Department of Computer Science

COMP4300- Graduation Project

Project Proposal 2025/2026

Section – A

Title of Project: Takkeh تَكّة

Group Members with IDs:

Enas Hamayel - 1210632
Roaa Ali Ahmad - 1210700
Nour Manasrah - 1211163

Supervised by:
Mr. Fadi Khalil

Section : 5

Graduation Proposal submitted to the Department of Computer Science in partial fulfillment of the requirements For the degree of B.Sc. in Computer Engineering
Jan 2026

Section – B

Title of Project: Takkeh تَكّة

Project No: 005

Supervisor: Mr. Fadi Khalil

Key Areas:

Artificial Intelligence (AI), Natural Language Processing (NLP), Real-time Data Collection, Social Media Integration, Crowdsourced Reporting, Mobile Application Development, Database, Geolocation Services, Security and Privacy, Localization, Scalability.

Section – C

Student Signature: _____ **Date Submitted**: _____

Student Signature: _____

Student Signature: _____

Supervisor Name: _____ **Supervisor Signature:** _____

Date Approved: ____ / ____ / ____

Abstract

Takkeh is a **smart traffic navigation application** that provides minute-by-minute updates on road conditions, including traffic congestion, checkpoints, roadblockages, and iron gates in Palestine. The system collects live data from social media platforms such as Telegram, in addition to user updates, to deliver accurate and timely road information.

What distinguishes Takkeh is its use of advanced natural language processing (NLP) to extract accurate information from unstructured public messages. Supported by a reliable database backend, the system delivers rapid, efficient notifications via an easy-to-use mobile interface.

By bridging the gaps left by current apps such as Waze [8] [10], "أحوال الطرق في فلسطين"[11]، "ع وين رايح"[12]"and Azmeh[13]. Takkeh provides a locally adapted, smart assistant that enables more efficient mobility. It combines contemporary technologies such as geolocation services, push notifications, and cross-platform app development, with an emphasis on privacy, scalability, and usability—positioning it as an essential tool for Palestinian commuters .

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

The proposal is spread over three chapters. In this chapter, we introduce the motivation background of Takkeh, the most challenging problems of road navigation in Palestine, and the objectives of the proposed system. Chapter Two introduces the background, relevant research, and literature review associated with technologies of similar systems. Chapter Three declares our proposed approach, system architecture, resources required, and the ethical issues involved.

1.1 Overview

In the modern day and age, precise and intelligent navigation systems are more crucial than they have ever been. Although there are many global programs like Google Maps and Waze that offer real-time routing, they are less localized in nature when it comes to accuracy and responsiveness, particularly in regions where the conditions of roads and checkpoints are complex, such as in Palestine. Journeys to the Palestinian areas are regularly interrupted by arbitrary roadblocks, military checkpoints, congestion, and destruction of infrastructure — issues requiring a solution particular to the local climate's situation. Takkeh is an intelligent traffic monitoring and advice system specifically designed for Palestine.

It applies real-time harvesting of information from public sources such as Telegram to provide its users with the current traffic and road conditions.What makes Takkeh unique is its integration with the Natural Language Processing, which intelligently processes unstructured text to pull out useful traffic information. With a database and other technologies, the system is designed to provide quick and local updates to drivers, minimizing the use of phones while driving and improving safety. The project is designed to bridge the gap between global technology solutions and local road navigation needs. Through the delivery of precise, real-time Palestine-specific traffic information, Takkeh seeks to enable commuters to make better-informed commuting decisions, promote road safety, and serve as a platform for smart, regional navigational systems.

1.2 Aim and Objectives

➢ Aim :

The aim of this project is to design and develop a smart traffic monitoring and navigation system. **Takkeh** provides real-time, accurate information about road conditions, traffic congestion, and checkpoints in Palestine. This will be achieved by utilizing data from Telegram Platform. The system will help users make better navigation decisions, reduce travel delays, and enhance safety through intelligent data analysis and timely notifications.

➢ Objectives :

1. **To collect live traffic-related data** from the Telegram platform.
2. **To filter and display checkpoints data based on user preferences** such as status (open/closed) and location (governorate and its villages), in order to enhance navigation and provide relevant traffic information.
3. **To store and manage the processed data** in a real-time database for efficient access and synchronization.
4. **To develop a user-friendly interface** that delivers road alerts through visual notifications.

Chapter 2: Background and Literature Review

2.1 Details of relevant theory

The Takkeh project is based on several important theoretical concepts. First, it utilizes crowd-sourced data collection, which involves gathering information from a wide range of users to deliver real-time updates on road conditions. This approach improves the accuracy and responsiveness of data, particularly in areas with sudden or unpredictable obstacles like roadblocks and checkpoints.

The project uses GPS technology to determine the user's location on the map and display the location of nearby checkpoints clearly.

Natural Language Processing (NLP) is essential to the Takkeh system's functionality as it enables the extraction of structured information from unstructured text sources such as social media messages.[1] In terms of the system implementation, our first step is to use APIs to ingest public road status messages from Telegram, while ignoring unnecessary messages. Then, we use native NLP algorithms to extract structured data (location, road status, timestamp) from the relevant messages[10].[2]

Furthermore, Real-Time Systems Theory underpins the system’s ability to deliver immediate notifications and real-time road status updates, ensuring that users receive information without delays during navigation.

Finally, Takkeh is designed using Human-Centered Design principles, focusing on usability and safety. Features such as an intuitive user interface are specifically developed to support drivers and reduce distractions on the road.

By integrating these diverse theoretical foundations, Takkeh offers a comprehensive, intelligent, and user-focused solution to the unique traffic challenges in Palestine.

2.2 Review of past/reported work

Navigation apps like Waze provide valuable real-time traffic data in many parts of the world. However, they often fall short in regions with complex and dynamic conditions, such as Palestine, where mobility is frequently disrupted by military checkpoints, roadblocks, and sudden closures. Our project addresses this gap by developing a smart road issues monitoring system that delivers real-time updates , aimed at improving both safety and driving efficiency.

Motivating Articles:

1.The Wall, Bypass Roads and The Dual Transportation System in Palestine

This study highlights the severe mobility challenges that Palestinians face as a result of the separation wall, bypass roads, and checkpoint systems. It underscores how these barriers fragment the road network and limit movement, especially in the West Bank. These findings strongly support the need for localized, real-time navigation solutions, which directly motivates the goals of our project. [3]

2.The Labor Market Impact of Mobility Restrictions: Evidence from the West Bank

This article analyzes how restricted movement in the West Bank negatively impacts employment opportunities by delaying commutes and reducing access to workplaces. It highlights the broader socio-economic consequences of inefficient road networks and strengthens the case for tools that enable faster, safer, and more informed mobility - a core objective of Takkeh. [4]

By reviewing past literature on transportation and mobility challenges in Palestine, we better understand the gaps in existing solutions and justify the development of a context-aware, intelligent navigation system tailored to local needs.

Previous works related to our project idea :

1- Waze: is a GPS navigation app that relies on community input to deliver real-time traffic updates and road information. Users contribute by reporting traffic jams, accidents, hazards, and police presence, allowing Waze to suggest alternative routes to avoid congestion. The app features voice-guided navigation and includes social elements for sharing locations and estimated arrival times. Additionally, it incorporates gamification, rewarding users for their contributions and regular driving [8] [10].

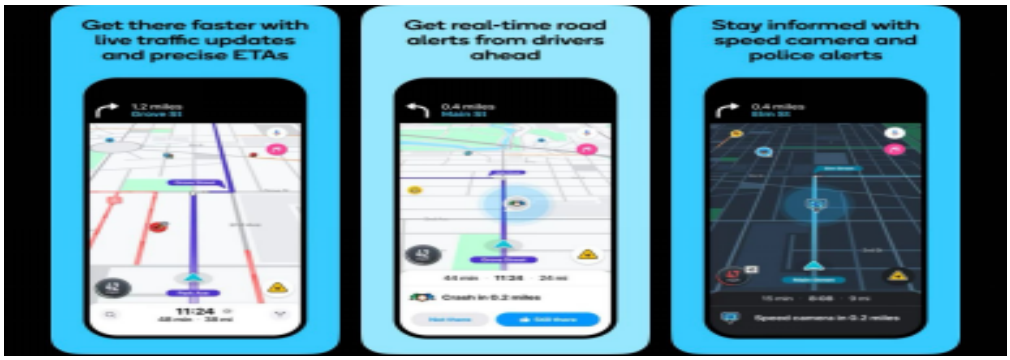

Figure 1:Waze Application Interface

2-The Palestine Road Conditions App "أحوال الطرق في فلسطين" : is a local navigation and traffic monitoring system designed to help drivers navigate roads efficiently. It provides updates on road closures, traffic congestion, and alternative routes, often relying on information from local authorities and reports from users[11].

Figure 2:أحوال الطرق في فلسطين Application Interface

3-Palestine Road Watch (ع وين رايح؟): Palestine Road Watch is a crowdsourced web application that helps users in the West Bank navigate a jammed terrain of road closures, checkpoints, and military blockades. The application allows users to file a report about road conditions, and view existing reports in real time about the conditions of roads, and provides situational awareness for drivers who travel between regions.Although Palestine Road Watch provides a valuable service to communities for sharing community verified information, the application is limited in its capacity to support automatic route planning, has artificial intelligence and does not connect with mapping APIs to enable dynamic navigation or alternative route options [12].

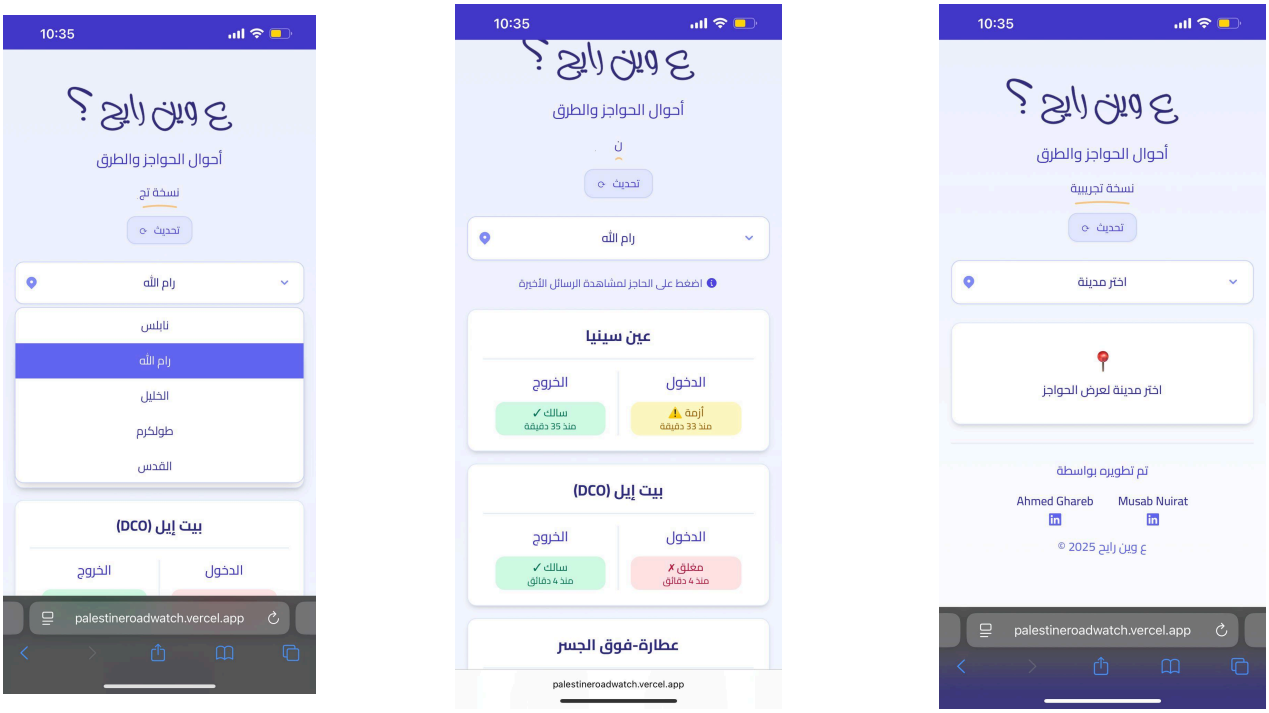

Figure 3: ع وين رايح ؟ Application Interface

Azmeh ” أزمة ”: is a localized traffic monitoring application that measures and visualizes urban congestion occurring in major Palestinian cities. The application accesses data from mobile users and traffic sensors to show the level of traffic flow where color-coded diagrams indicate traffic flow on roads. The application aims to reduce travel delays and increase urban mobility by warning users of high-congestion areas, although it does not report security-related obstructions or roadblocks that render it less informative than similar applications in inter-city or rural travel, where security threats are prevalent [13].

Figure 4: أزمة Application Interface

Comparison of existing applications and what can be developed in the project :

Feature	Takkeh (Our App)	Palestine Road Watch ("ع وين رايح؟")	Azmeh	Istilamat Falasteen (استعلامات فلسطين)	Waze
Real-Time Road Updates	Combines user updates on the road's status, AI, and live sources like Telegram	Relies on manual updates by the admin team	Manual updates from users only	Not supported	Fully user-driven live reports
Checkpoint and Roadblock Availability	Roadblocks and checkpoints displayed directly	Roadblock info extracted automatically from Telegram channels	User-added checkpoints manually	Not designed for live road conditions; focuses on public service inquiry .	Limited or no checkpoint info in West Bank context
Offline Usage Support	Cached data enables offline access by showing the last update before losing the connection	Requires internet	No offline capabilities	Not applicable	Requires constant internet
AI-Powered Data Processing	Uses AI algorithms, specifically Natural Language Processing (NLP), to analyze and extract information from live text sources	Not available	Not available	Not available	Not available

Map Visualization	Interactive map with smart icons (checkpoints and user's location)	Basic map with route alerts	Static map visuals	No map integration	High-quality, interactive map
Language Support	Arabic only	Arabic only	Arabic only	Arabic only	Multiple languages (English, Hebrew, etc.)
Automatic Data Sync	Real-time updates via bots/APIs	Manual data entry	Community-only	No sync mechanism	Yes, via user reports
Local Road Data Accuracy	Continuously updated from multiple local sources	Moderate – team-based updates	Varies – depends on user input	Low – not intended for navigation	Limited – less reliable for local Palestinian roads
Smart Notifications	Alerts when the road status changes for a subscribed city	None	None	None	None for roadblocks/checkpoints
Login & Personalization	Supports login, notification preferences	No user account system	No	No	Google-based account required

Table 1: Comparison table between Takkeh and other applications

2.3 Tools and Technology

The development of Takkeh relies on modern tools and technologies that support real-time traffic monitoring, intelligent data processing, and cross-platform mobile development. Below is an overview of the main data sources and tools used in the system:

Data Sources:

- **Telegram [14]:** Primary source for real-time road and checkpoint updates shared by local communities.

Tools & Technologies used:

1.Database: The Takkeh system uses a hybrid database approach. We rely on **Supabase[6]** to store and manage all the main application data, such as user information, checkpoint details, road statuses, subscriptions, and admin-related actions. This allows the system to keep the data organized and quickly accessible.

In addition, **Firebase [7]** is used to handle push notifications for both users and admins. It enables the system to send real-time updates and alerts whenever important changes occur. By combining both technologies, the system achieves reliable data management along with fast, real-time communication.

2.External APIs: Road updates are collected from public sources, including Telegram platforms, utilizing accessible APIs.

3.Natural Language Processing (NLP): After getting raw text data from outside sources, the system first filters the messages to remove irrelevant content. Only the relevant traffic-related texts are then sent to the internal NLP algorithms, which analyze and extract structured information like road status, location,direction and event time. [5]

4.Mobile Development Framework: like Flutter, Supports cross-platform app development for Android and iOS with a unified codebase.

5.GPS & Mapping Services: GPS is used only to determine the user's current location and show checkpoints on the map interface.

Key Outcomes of Using These Technologies: Real-time and localized traffic alerts. Analysis of user reports and public posts. Community-driven data enrichment through user submissions. Seamless user experience through a responsive and intuitive mobile app.

Chapter 3: System Analysis and Design

3.1 Product Description

Takkeh is an innovative mobile application tailored to assist drivers in Palestine with safe and efficient navigation. It offers real-time traffic information, including updates on road closures and checkpoints, by aggregating real-time data from public Telegram channels. The application employs artificial intelligence, specifically NLP, to process unstructured text and extract pertinent traffic details. This information is stored in a database for fast retrieval and is communicated to users through visual alerts, hence minimizing distractions for drivers. Designed to meet local requirements, Takkeh includes features such as offline functionality (Relies on the last received update before internet disconnection to provide critical information during offline usage), and an intuitive interface to enhance the commuting experience in challenging road environments.

Roles and users in systems:

There are three basic categories of users in the system:

New User: A new user needs to finish a registration process to set up an account before they can use the app.

Registered User: Views road conditions, updates road status and gets alerts.

Admin User A user who monitors the application, verifies user-submitted updates by approving them and can manually change the status of a checkpoint (such as open or closed).

3.1.1 System Objectives

1. Gather real-time traffic information from Telegram.
2. Store processed data in the database for real-time accessibility and refresh.
3. Generate visual notifications to the users to avoid distraction while driving.
4. Provide offline access to the last available updates.
5. Enable users to interact with the application by using features like updating road

conditions, subscribing to certain areas, and getting tailored traffic updates for chosen area.

6. Test and analyze the system's performance, usability, and reliability in real-world scenarios.

7. Automatically remove outdated updates: Traffic updates older than 7 days are automatically deleted from the database to maintain data relevance and optimize storage.

3.1.2 System Main Features

1. **Real-Time Road Monitoring:** Live updates on road conditions, traffic, accidents and checkpoints.

2. **Text Analysis:** Uses NLP Algorithms to extract traffic data from filtered unstructured text like Telegram groups / supergroups messages.

3. **User Interaction:** Allows users to contribute by updating road statuses, subscribing to specific areas, and receiving tailored traffic updates.
4. **Interactive Map:** Visual display of road checkpoints and user's location .
5. **Push Notifications:** Instant notifications are sent to users & admins when the road status changes for any city they are subscribed to.
6. **Data Validation & Filtering:** Ensures reliable, relevant traffic information.
7. **Automatically remove outdated updates:** Traffic updates older than 7 days are automatically deleted from the database to maintain data relevance and optimize storage.
8. **Resilient Offline Operation Mode:** The application is designed to support reliable operation even during periods of limited internet or power connectivity, allowing cached information to remain available.
9. **Battery Consumption Efficiency:** Takkeh is optimized to reduce battery usage by minimizing background GPS requests and notifications, using event-driven logic and region-based updates instead of constant polling.

3.1.3 Operating Environments

1. **Mobile Platforms:**

   The system is engineered to function on Android and iOS devices, utilizing cross-platform frameworks such as Flutter or Android Studio.

2. **Backend and Database:**

   The backend utilizes the database to handle user data and traffic details, providing efficient storage and real-time synchronization via the application server.

3. **Natural Language Processing (NLP):**

   The system employs internal NLP algorithms to analyze and extract structured road information from unstructured textual data collected from traffic-related sources.

4. **Internet Connectivity:**

   An internet connection is necessary for real-time updates, although offline access to previously synchronized data is supported.

5. **Supported APIs & Tools:**

   The system gathers traffic updates by connecting directly to the official APIs of Telegram . This guarantees dependable and organized data collection. Furthermore, push notification services are utilized to send prompt alerts to users.

6. **Backend Automation:** The backend periodically detects and deletes outdated traffic updates (older than 7 days).

3.1.4 Constraints

1. **User Updates Dependency:**

   The system relies on user-submitted traffic updates, which may be inconsistent or unavailable in some areas.

2. **Limited User Engagement:**

   Without strong incentives, maintaining active participation may be difficult.

3. **Language Challenges:**

   Variations in Arabic dialects and spelling may reduce the accuracy of text interpretation.

4. **Map Data Limitations:**

   Some areas may lack detailed or updated map information.

5. **Internet Connectivity:**

   Unstable internet access in certain regions can affect app performance.

6. **Privacy Concerns:**

   Handling user location and activity data requires strong privacy measures.

7. **Resource Limitations:**

   Development was constrained by time, tools, and team size compared to commercial apps. Additionally, it was challenging to collect data from platforms like Facebook and Instagram due to strict data access restrictions and platform limitations.

3.1.5 Functional Requirements

User Requirements:

UR 1: The system shall allow users to register using their email address.

UR 2: The system shall allow users to log in using their registered credentials .

UR 3: The system shall provide users with relevant traffic updates by processing information received from messaging platforms.

UR 4: The system shall store all validated traffic and incident information in a database in real-time.

UR 5: The system must provide real-time information and notifications that are relevant to any subscribed checkpoints

UR 6: The system shall produce a live map displaying the user's current location and the locations of checkpoints.

UR 7: The system shall allow users to search specific roads and checkpoints .

UR 8: The system should be tested in real-world usage contexts with varied user groups to ensure efficacy and usability.

UR 9: The system shall allow administrators to manually update the status of a road .

UR 10: The system shall allow registered users to update on the status of a road.

UR 11: The system shall allow users to filter the data shown.

UR 12: The system shall allow an administrator to nominate a registered user for administrator privileges.

UR 13: The system shall allow administrators to monitor the performance of the NLP processing pipeline and user activity within the system.

UR 14: The system shall allow registered users to reset their password if they forget it.

R15: The system shall allow users to contact the system administration via an official email address displayed on the login interface.

System Requirements:

UR 1: The system shall allow users to register using their email address.

SR 1.1: The system shall provide a registration interface that allows users to create an account using their email address by first entering a valid and unique email address.

SR 1.2: The system sends a secure email verification link.

SR 1.3: The user completes the verification by opening the link and confirming the action inside the application.

SR 1.4: After successful email verification, the system shall display a second screen where the user is required to create a unique username, a secure password, and confirm the password.

SR 1.5: The system shall store the registered username, password and verified email address securely in the database.

UR 2:The system shall allow users to log in using their registered credentials.

SR 2.1: The system shall provide a login interface that allows users to enter their username/email address and password.

SR 2.2: The system shall validate the entered credentials by checking them against the stored user data in the database.

SR 2.3: If the authenticated user exists only in the users table, the system shall redirect the user to the User Dashboard.

SR 2.4: If the authenticated user exists in both the users table and the admin table, the system shall prompt the user to choose between entering as a User or as an Admin.

SR 2.5: Based on the user's selection, the system shall redirect the user to the corresponding dashboard (User Dashboard or Admin Dashboard).

UR 3: The system shall provide users with relevant traffic updates by processing information received from messaging platforms.

SR 3.1: The system should be linked to a specific Telegram group / supergroup from which traffic messages will be received via their official APIs.

SR 3.2: The system shall retrieve messages from Telegram groups / supergroups having unstructured filtered text and process the text.

SR 3.3: The system shall recognize and extract traffic related content from Telegram messages by scanning a list of keywords and patterns, and applying rule-based NLP algorithms (including text normalization, gazetteer-based location extraction, direction detection, and weighted pattern matching) to identify traffic-related entities such as city, status, and directions.

SR 3.4: The system must categorize the information retrieved from the messages into generic traffic reports like accidents, road closures and traffic jams, using rule-based classification with confidence scoring, also it must recognize specific categories like security incidents or roadway obstruction due to congestion.

SR 3.5: The system shall analyze traffic reports for essential content such as city, status, direction and time stamp.

SR 3.6: The organized output shall be held in the database for the purpose of alerting and searching and monitoring routes.

SR 3.7: The system should display the analyzed traffic reports on the app UI in an understandable and satisfactory format.

SR 3.8: The system shall give users the option to view the latest traffic reports, and filter by location and/or category.

SR 3.9: The system shall maintain duplicate copies of the message related to

traffic from Telegram groups/supergroups to provide cross-checking and higher confidence of the identified incident.

UR 4: The system shall store all validated traffic and incident information in a database in near real-time.

SR 4.1: The system shall establish a secure connection with a database for storing validated traffic and incident information in real-time.

SR 4.2: The system shall ensure that each stored traffic report in the database includes details such as the location, status, direction and timestamp.

SR 4.3: The system shall implement authentication and authorization mechanisms to restrict database access, ensuring that only authorized users or services can read or write data.

SR 4.4: The system shall maintain data integrity and availability in the database through automatic backup and error-handling mechanisms at the server level.

SR 4.5: The system shall manage database storage by automatically removing outdated traffic reports based on a predefined retention policy.

UR 5: The system must provide real-time information and notifications that are relevant to any subscribed checkpoints.

SR 5.1: The system shall allow users to subscribe to one or more checkpoints to receive status updates and select a subscription duration that suits their needs (e.g., hours, days, or custom period)

SR 5.2: The system shall monitor the status of all checkpoints in real-time and detect any changes like from open to closed or vice versa for both the outside and inside.

SR 5.3: The system shall automatically send a real-time notification to users when the status of a subscribed checkpoint changes.

SR 5.4: The system shall allow users to manage their list of subscribed checkpoints through the application interface.

SR 5.5: Notifications about checkpoint changes shall be delivered using push notifications and in-app alerts for better visibility.

SR 5.6: The system shall ensure that checkpoint change notifications are sent only to users who have subscribed to the affected checkpoint.

UR 6: The system shall produce a live map displaying the user's current location and the locations of checkpoints.

SR 6.1: The map should display the current location of the user using integrated Maps APIs.

SR 6.2: The map shall display **checkpoints and road locations as map markers**, where each marker visually represents the **current road status** using distinct colors and icons.

SR 6.3: The users should be able to navigate the map by zooming in or zooming out, to different areas and updates on the map.

SR 6.4: The system shall allow users to **filter displayed map markers** based on road status categories such as open, closed, traffic congestion, accidents, and unclear conditions.

SR 6.5: The system shall provide a **search feature** that enables users to quickly locate cities or checkpoints on the map.

UR 7: The system shall allow users to search specific roads, checkpoints.

SR 7.1: The system must provide a search interface where users can enter keywords relating to road names or checkpoints.

SR 7.2: The system must retrieve related results from the database based on the search query entered by the user.

SR 7.3: The system shall show the search results in a list containing the checkpoint name , the status in each direction and the last update time.

SR 7.4: In case there are no results, the system shall inform the user with a message indicating no matches.

UR 8: The system should be tested in real-world usage contexts with varied user groups to ensure efficacy and usability.

SR 8.1: The system should be accessed by at least 3 distinct user groups: novice, intermediate, and advanced.

SR 8.2: Testing usability in real life must be carried out (such as driving, congestion, low range/low connectivity).

SR 8.3: Feedback from users must be gathered from such real-world testing via structured questionnaires and interviews.

SR 8.4: The results of the usability tests must be analyzed to identify usability problems, performance bottlenecks and other areas for improvement.

SR 8.5: The system shall be incrementally optimized and enriched to comply with the true usage protocols acquired under the pre-final rollout testing.

UR 9: The system shall allow administrators to manually update the status of a checkpoint.

SR 9.1: The system shall provide an admin dashboard where the administrator can view a list of all active checkpoints.

SR 9.2: The admin dashboard shall allow the administrator to select a checkpoint and update its status .

SR 9.3: The system shall log all administrative changes to checkpoint status for auditing purposes , it will be recorded as (adminId, adminName, checkpointId, checkpointName,direction, oldStatus, newStatus, timestamp)

and will be shown in the Update History dashboard.

SR 9.4: The updated status shall be reflected in the app UI and push notifications in real-time.

UR 10: The system shall allow registered users to update on the status of a road .

SR 10.1: Registered users shall be able to update a checkpoint's current condition/status through a "اقتراح تغيير حالة الطريق" button associated with each checkpoint on the app.

SR 10.2: The system shall store each update with metadata including user ID, checkpoint ID, direction, new status, and timestamp.

SR 10.3: If five or more updates are received from different users for the same checkpoint with the same suggested status change within a 30-minute window, the system shall automatically update the checkpoint status.

SR 10.4: Alternatively, if the admin approves a user-submitted update manually, the system shall immediately update the checkpoint's status accordingly.

SR10.5: The system shall prevent duplicate submissions by the same user for the same checkpoint status change within a limited time period like 30 minutes.

UR 11: The system shall allow users to filter the data shown.

SR 11.1: The system shall provide a filter interface allowing users to select filtering criteria relevant to the data context

SR 11.2: The system shall apply the selected filters to display only matching data

SR 11.3: The system shall allow multiple filters to be combined for more precise results.

SR 11.4: The system shall provide clear options to apply, reset, or clear filters.

SR 11.5: If no results match the filters, the system shall notify the user with an appropriate message.

UR 12: The system shall allow an administrator to nominate a registered user for administrator privileges.

SR 12.1: The system shall allow an administrator to submit an admin nomination request for a selected registered user.

SR 12.2: The system shall send a notification to the nominated user indicating that an administrator has requested to grant them administrator privileges.

SR 12.3: The system shall allow the nominated user to **accept or reject** the admin nomination request.

SR 12.4: If the nominated user accepts the request, the system shall grant admin privileges by adding the user to the admins table.

SR 12.5: If the nominated user rejects the request, the system shall keep the user as a regular user and mark the request as **rejected** with no changes to admin privileges.

SR 12.6: The system shall allow administrators to view the nomination request status for monitoring purposes.

UR 13: The system shall allow administrators to monitor the performance of the NLP processing pipeline and user activity within the system.

SR 13.1: The system shall allow administrators to view statistics related to NLP message processing, including processed, saved and failed messages.

SR 13.2: The system shall allow administrators to view confidence-related metrics for extracted road statuses.

SR 13.3: The system shall allow administrators to monitor NLP processing errors and skipped messages.

SR 13.4: The system shall allow administrators to view aggregated user activity within the system.

SR 13.5: The system shall display monitoring information through the Admin Dashboard.

UR 14: The system shall allow registered users to reset their password if they forget it.

SR 14.1: The system shall provide a "Forgot Password" option on the login screen.

SR 14.2: Upon selecting "Forgot Password", the system shall display a screen that allows the user to enter their registered email address.

SR 14.3: The system shall verify that the entered email address exists in the database

SR 14.4: If the email address is valid, the system shall send a secure password reset link to the entered email address.

SR 14.5: The system shall allow the user to open the reset link and access a secure web page to enter and confirm a new password.

SR 14.6: The system shall enforce password security requirements before updating the password.

SR 14.7: After successful password reset, the system shall redirect the user to the login page.

UR15: The system shall allow users to contact the system administration via an official email address displayed on the login interface.

SR 15.1: The system shall display the official administration email address

prominently on the login screen.

SR 15.2: The system shall allow users to click on the displayed email address, which opens a new email message in the user's default email client with the recipient field pre-filled with the administration email and the subject pre-filled as “Inquiry from Takkah App”.

SR 15.3: The system shall ensure the displayed email address is valid and reachable.

SR 15.4: The system shall provide brief instructions or a tooltip indicating that the email can be used to contact administrators for support or inquiries.

3.1.6 Non-Functional Requirements

Performance:

The application must synchronize data in real-time and provide low-latency updates and notifications. Search queries should return less than 5 seconds of results under normal loads.

Scalability:

The application must be able to handle increasing numbers of users and data inputs without performance degradation. The backend infrastructure must be scalable, and utilize cloud services.

Usability:

The application interface must be simple and intuitive, in that any fundamental feature can be accessed after no more than 3 steps. The interface should be accessible to everyone of every generation, and technical skills.

Additionally, users shall be able to submit traffic updates through a straightforward updating feature designed to encourage participation.

□ Reliability:

The system must have high availability, in that it never drops below 99% this is over a calendar month, to provide uninterrupted service.

□ Maintainability:

The system's architecture must be modular so updates and deployment of new functionality is easy and does not require large system reworks. Full technical documentation must be included as part of developer and maintainer support.

□ Compatibility:

The application must operate on both Android and iOS platforms. It should function properly across a wide range of screen sizes and layouts, ensuring consistent performance and usability on all supported mobile devices.

□ Security:

The system must offer safe authentication and encrypted data transfer. Users' location and personally identifiable information must be stored securely in accordance with user privacy standards and prevailing legislation.

3.2 Functional Decomposition

3.2.1 Actors

The below table describes all actors engaged with the system:

Actors	Description
New User	A user who uses the app for the first time, and must register using an email address, username, and password to gain access to the app feature.
Registered User	A person who uses the application to log in, receive alerts, view road updates and report incidents or emergencies.
Admin	A system administrator who manages user data, verifies reports, monitors system performance, and maintains the integrity of the application.
Telegram Channels	External data sources that provide live updates about roads and emergencies from public or dedicated channels.
GPS	A location-based service that provides real-time positioning data to support navigation, user location detection, and context-aware notifications.
Email Verification Service	An external service responsible for sending email verification links to users during the registration process. The service ensures email ownership by redirecting users back to the

	system after successful verification.

Table 2: Actors Description

3.2.2 Use Cases (Summarization)

Use-Case 1: User Registration and Authentication

This use case describes how users register and log in to the Takkeh application using email-based verification. New users must verify their email address through a verification link before completing their registration by creating a username and password. All users are initially registered as regular users. Administrative users are identified based on their presence in the admin table after being promoted from a regular user account.

Use Case 2: Updating Road Status

This use case describes how a registered user can update the road's status such as open, closed, accident,checkpoint and traffic. Once five unique users submit the same updated status for the same road within a time period of 30 minutes or less, the system auto-updates the status. Alternatively, an administrator can directly approve user updates or manually change road statuses.

Use-Case 3: Searching for Specific Roads or Checkpoints

This use case describes a registered user interacting with the system to search for information about specific roads or checkpoints. The system processes the input, fetches data from the database, and returns relevant results. It also handles cases when no results are found.

Use-Case 4: Telegram Road Status Processing with NLP

In this use case, the Takkeh system automatically collects real-time road status updates from designated Telegram groups/supergroups. These messages are initially saved as

raw, unprocessed data and then analyzed using a local rule-based Natural Language Processing (NLP) pipeline.

The NLP module applies a combination of text normalization, gazetteer-based location extraction, rule-based status classification, direction detection, and historical pattern enhancement to convert unstructured Telegram messages into structured road status reports.

The extracted information—such as checkpoint location, road status, direction (inbound/outbound/both), confidence score, and timestamp—is stored in the database and reflected in the Takkeh mobile application as real-time road update.

3.2.3 Use Case Diagram

You can access the image by the following URL
https://drive.google.com/file/d/1b81fD950WSdk-XFGkx7d5_FVc8OBlS2E/view?usp=sharing

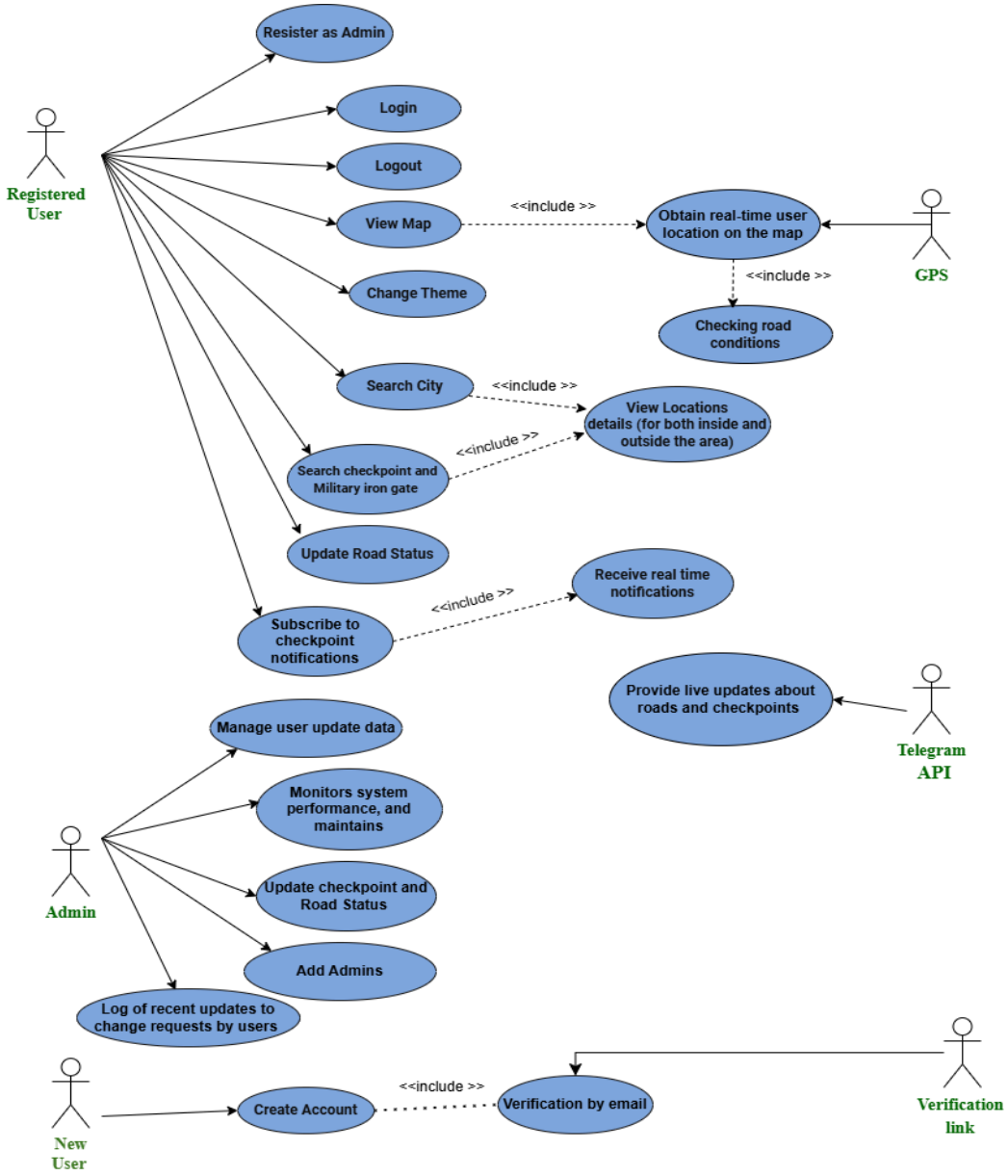

Figure 5: Use Case Diagram

3.3 System Models

System Analysis Classes

Below the identification of the system classes , describing each class. (Detailed Class Diagram).

User: User class represents a regular system user that is a main actor in interactions with the application . It includes such mandatory fields as userId, username, email, password and isAdmin flag . It allows users to sign up and login using secure login methods provided by the AuthenticationService with verification link. Once logged in, users may search for locations and filter based on traffic reports, as well as view current road and checkpoint conditions . Also, users can report road conditions and are able to vote in a process in which checkpoint statuses can be modified. Users also have the option to save specific checkpoints so they will receive alerts when a road's status alters. The isAdmin() function is employed to determine if a user has admin authority.

Admin: The Admin class is a subclass of the User class, inherits all user functionalities and adds some advanced admin features.

Admins approve or dismiss road updates and votes from users. They can confirm or reject any proposed changes, update road statuses manually and use the method confirmNewAdmin() to confirm administrator promotion requests from regular users. Additionally, administrators process the conflicting or invalid data for preservation of system consistency and integrity.

Subscription: A user- and checkpoint-related class is also the Subscription class. The information which is stored in it is SubscriptionId CheckPointID UserID Duration so on and so forth.

This class exposes methods to enable or disable subscriptions and return whether a subscription is expired. Subscriptions allow to have the information about the traffic of

some checkpoint automatically sent.

Notification: The Notification class is responsible for sending alerts out to users that subscribes to/news alerts on the respective checkpoint. Notification class is responsible for managing its notificationId, message, time, subscribed checkpoint, and subscription duration for notification subscription. When a checkpoint is updated a corresponding notification is sent to users using notifyIfStatusChanged().

AdminNotification: The AdminNotification class handles system-generated alerts directed specifically to administrators. These notifications inform admins about events such as pending user votes, suggested road status changes, automatic system updates, or user-generated reports.

Each admin notification contains metadata including checkpoint information, previous and new statuses, user identifiers, timestamps, and message content. Admins can mark notifications as read after reviewing them, ensuring efficient administrative workflow and oversight.

CheckpointStatusVote: The CheckpointStatusVote class implements the voting mechanism that allows users to propose changes to road or checkpoint statuses. Each vote records the userId, checkpointId, proposed status, traffic direction, creation time, and current review state (pending, approved, or rejected).

Votes are reviewed by administrators, who can approve or reject them. This mechanism supports community-driven updates while maintaining data validation through administrative control.

GPS: The GPS class collects the user's geographical location at the time of acquisition with

latitude and longitude. The GPS class contains the main methods viewRealTimeLocationOnMap() and viewCheckpointsOnMap(), which support location orientated interactions on the map and filtering.

TelegramChannel: They are external data sources that the system utilizes in order to automatically retrieve raw traffic messages. Each channel features a unique channelId, name, and link.

This source provides real-time textual traffic updates, which are later processed by the system's data pipeline to extract useful traffic information.

DataSource: This is the origin of the raw road messages being retrieved from platforms like Telegram.Data source is given a rawMessage, details on where the road was located such as, city,checkpoint state and it enables message parsing, validation, and assignment before passing data to the filtering stage.

DataFilter: This is responsible for clearing/cleaning and filtering raw messages coming into the system. DataFilter removes irrelevant messages or non-road-related messages from being classified. DataFilter ensures that only those messages pertaining to traffic are analyzed and stored.

DataNormalization (NLP): applies natural language processing techniques to interpret filtered messages. It converts unstructured human-written text into standardized traffic statuses such as Open, Closed, Accident, Checkpoint or Traffic , This step ensures consistent data representation across the system.

UpdateData: Is the main processor of all collected data. It receives the filtered raw text data as provided by DataFilter, applies the classification rules defined in the NLP module in the previous phase, and produces structured and categorized data records. This data is then used to be input to the back-end database at which point notifications are generated to users.

City / Road Status: Represents checkpoints and road segments. It stores attributes such as location identifiers, names, traffic status, confidence level, update time, and data source.

It supports query operations to retrieve current traffic conditions and historical updates.

Database: Acts as the persistent storage backend of the application. Manages the connection status, stored valid and processed location data, and retains only the most recent valid data. It identifies storage through explicit validation logic.

AuthenticationService: Acts as an intermediary layer between users and the database. It is responsible for handling identity verification processes such as sending and validating secure email-based verification links, as well as encrypting passwords. This ensures the security of user data during login and account creation operations.

3.3.1 Class Diagram

You can access the image by the following URL
https://drive.google.com/file/d/1xtMPIBmpm00OwxTqnQcJW87Ntk9m6N1z/view?usp=sharing

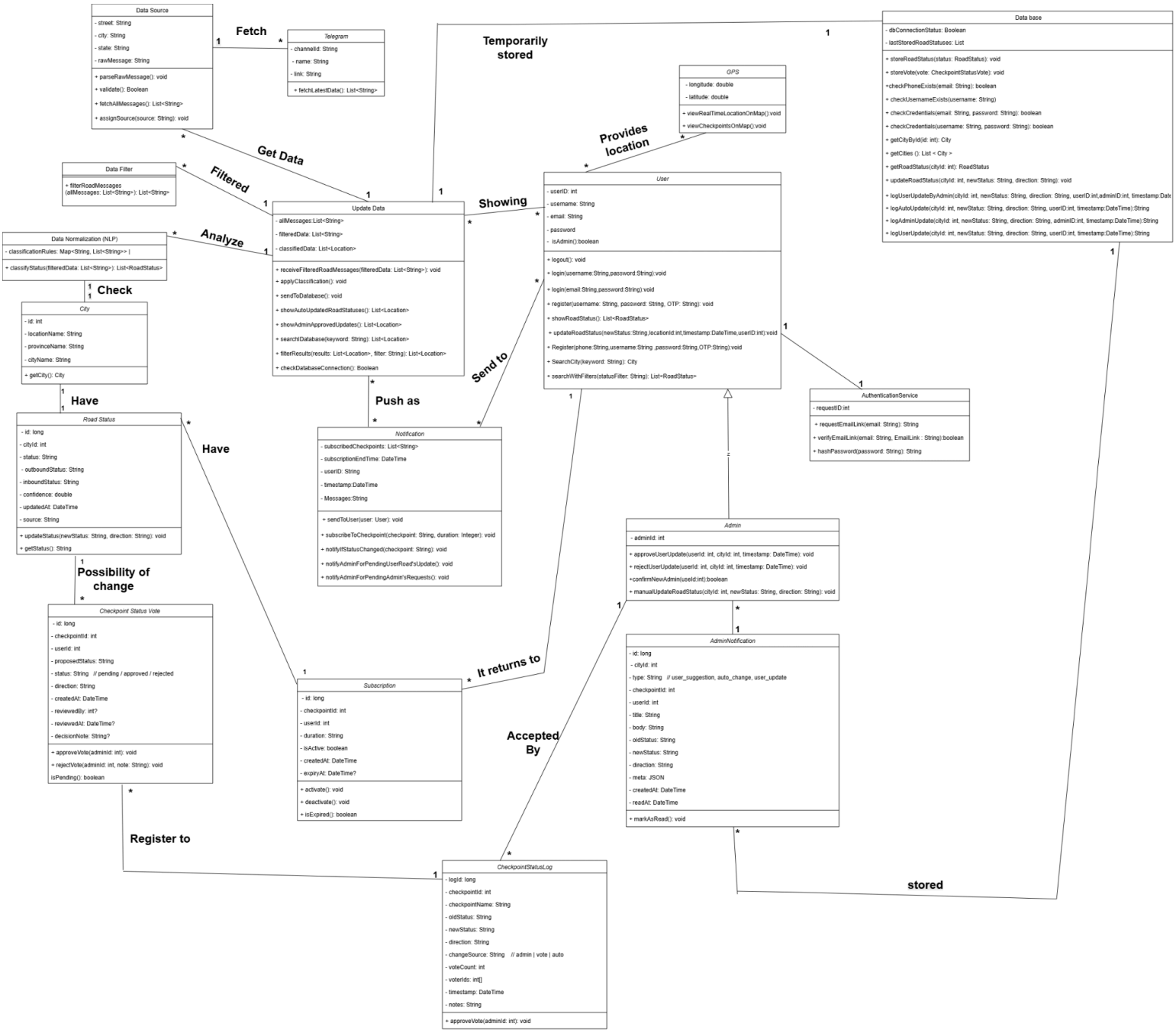

Figure 6: Class Diagram

3.3.2 Sequence Diagram

User Registration, Login and Authentication Sequence Diagram

You can use the following URL to access the image
https://tinyurl.com/2cb82hzs

Figure 7: User Registration and Authentication Sequence Diagram

Submit and Update Road Status Sequence Diagram

You can use the following URL to access the image

UpdateRoadStatus

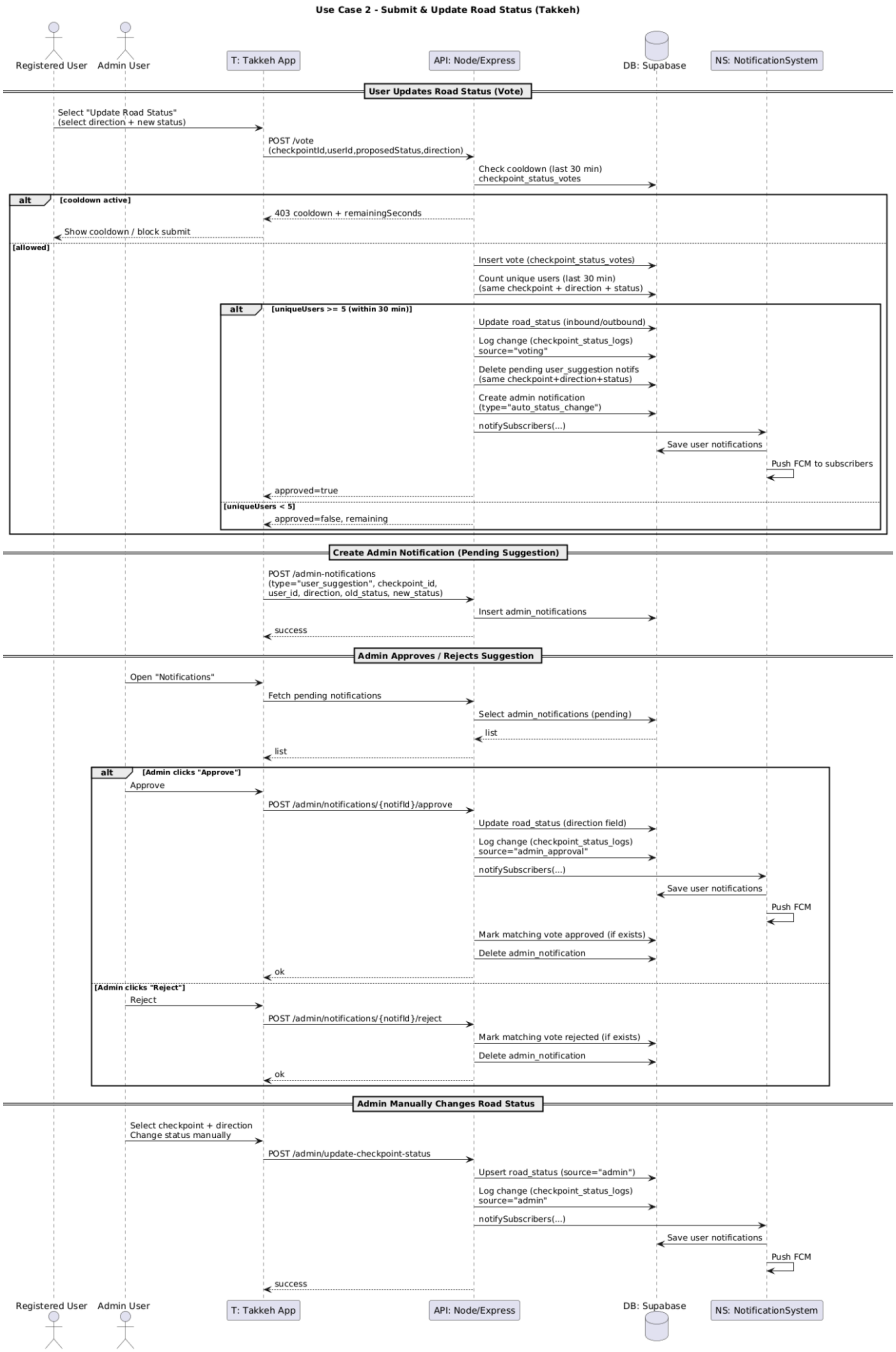

Figure 8:Submit and Update Road Status Sequence Diagram

Searching for Specific Roads Status Sequence Diagram

You can use the following URL to access the image
https://tinyurl.com/yvx67t9n

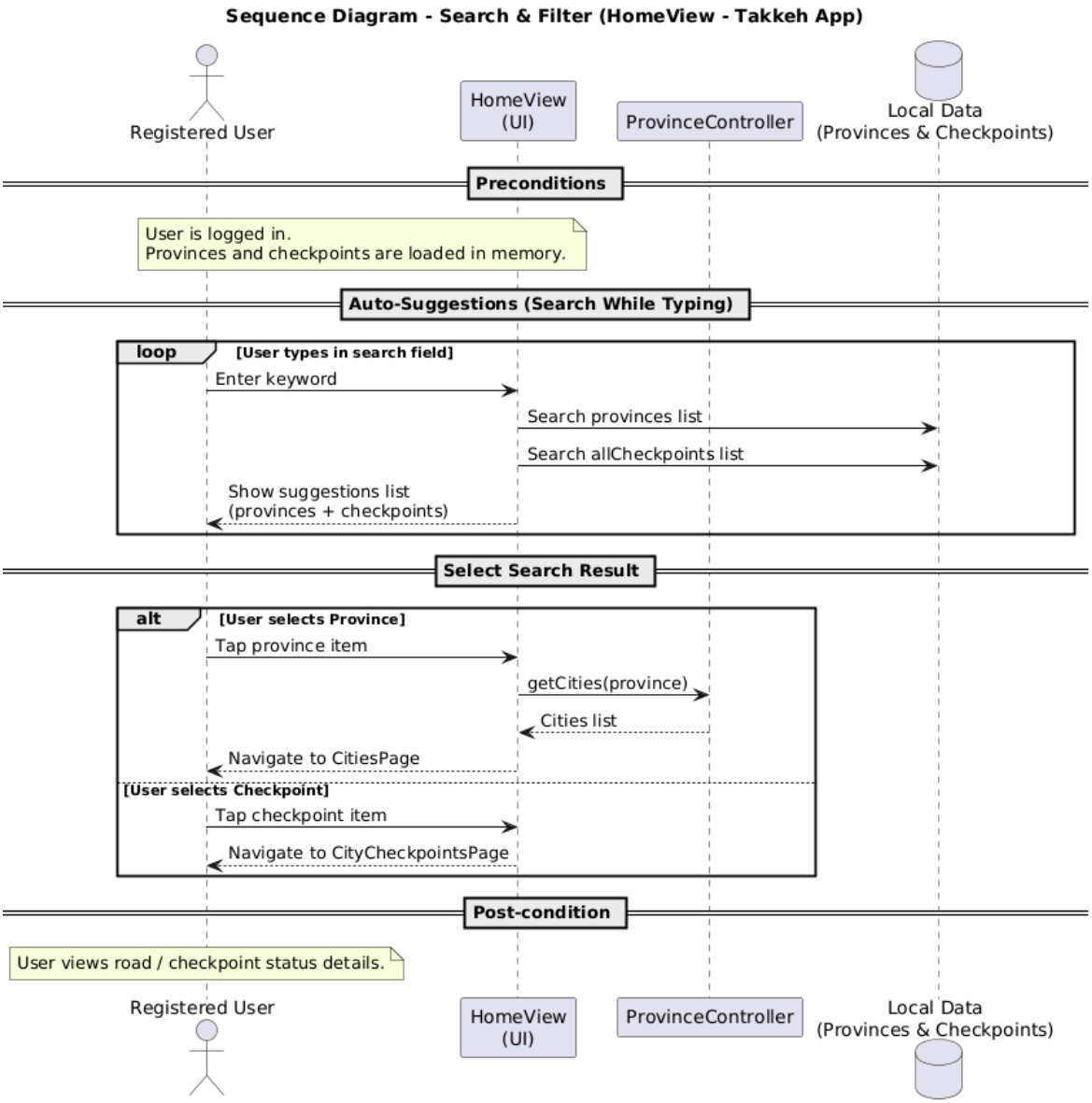

Figure 9:Searching for Specific Roads Status Sequence Diagram

Telegram Road Status Processing with NLP Sequence Diagram

You can use the following URL to access the image
NLP

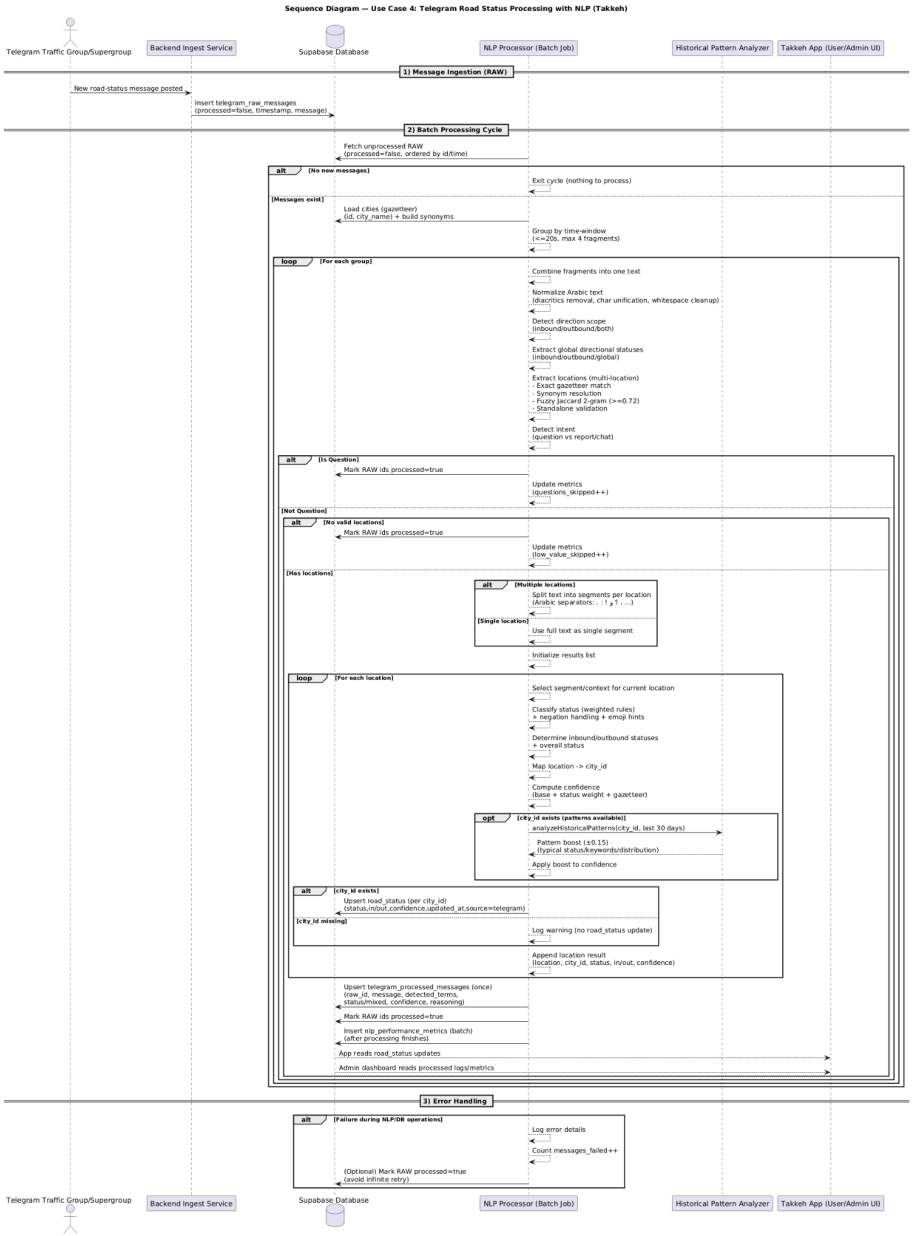

Figure 10:Telegram Road Status Processing with NLP Sequence Diagram

3.3.3 Activity Diagram

User Registration, Login and Authentication Activity Diagram

You can access the image here:
User Registration and Authentication Activity Diagram

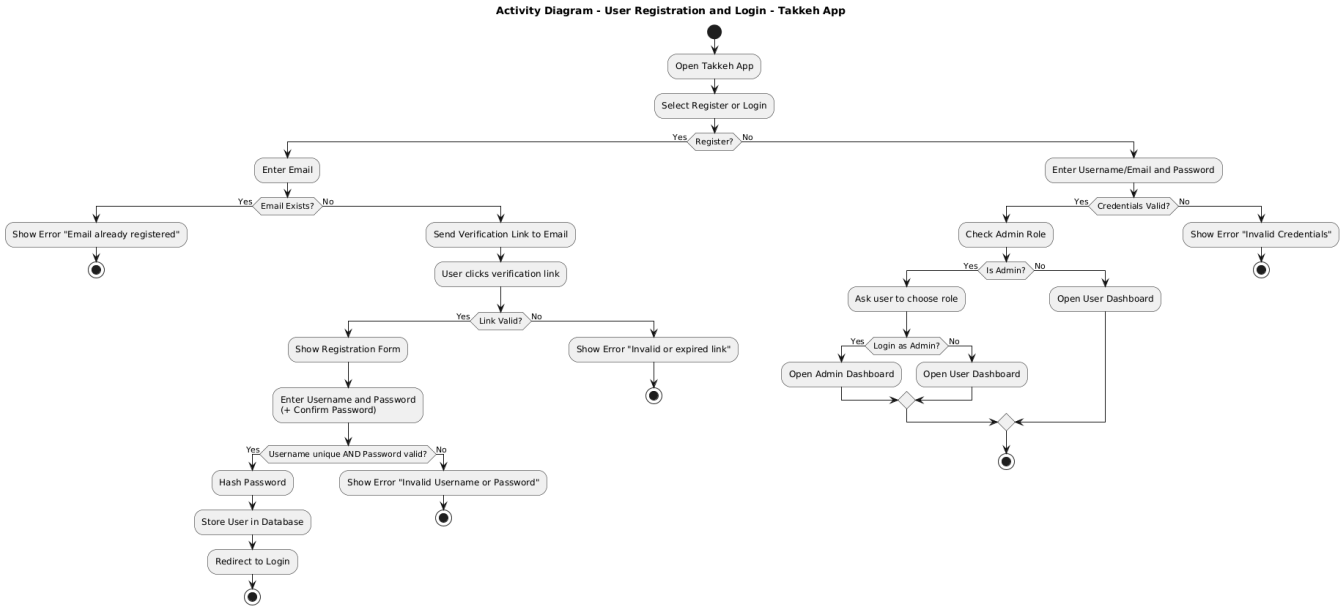

Figure 11:User Registration and Authentication Activity Diagram

Submit and Update Road Status Activity Diagram

You can access the image here:
UpdateRoadStatus

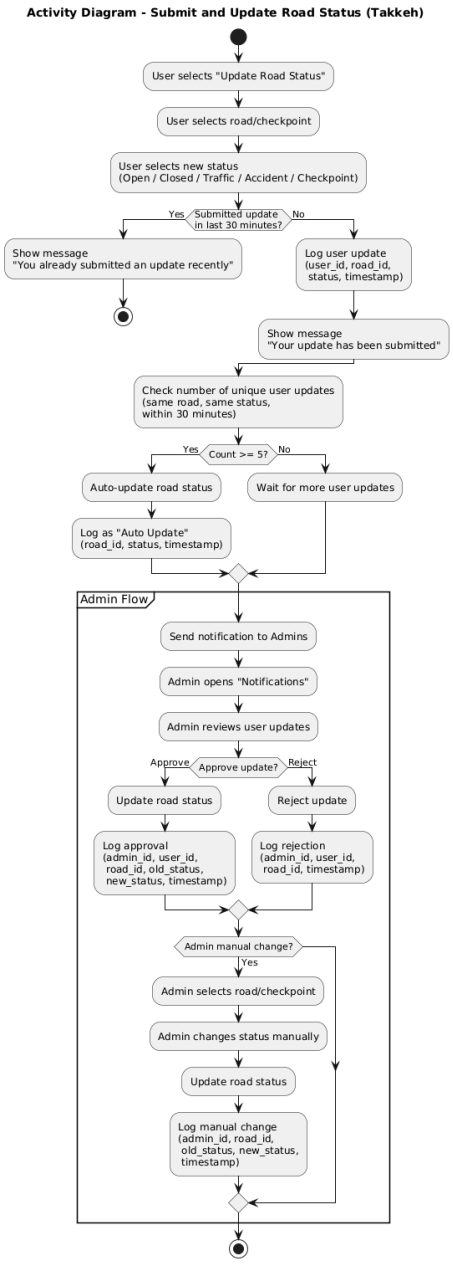

Figure 12:Submit and update Road Status Activity Diagram

Telegram NLP Processing Activity Diagram

You can access the image here:
NLP

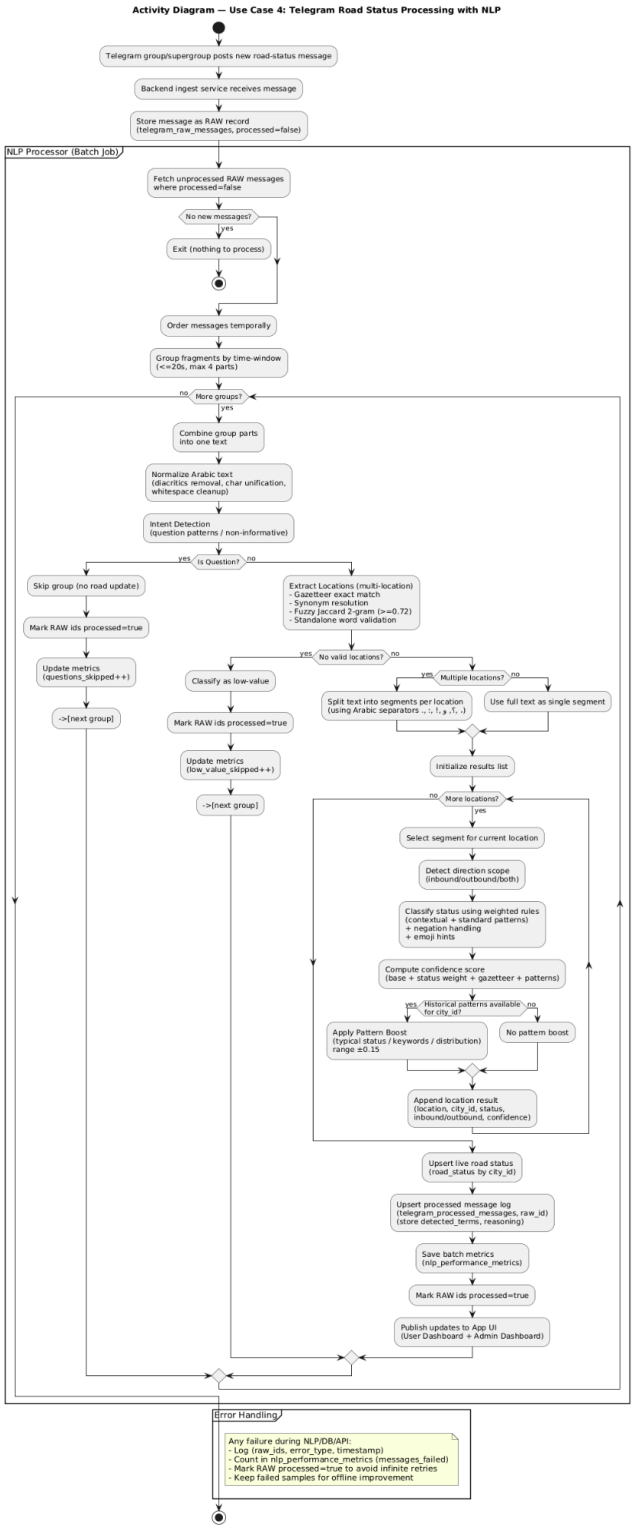

Figure 13:Telegram Traffic Report Processing with NLP Activity Diagram

Searching for Specific Roads Status Activity Diagram

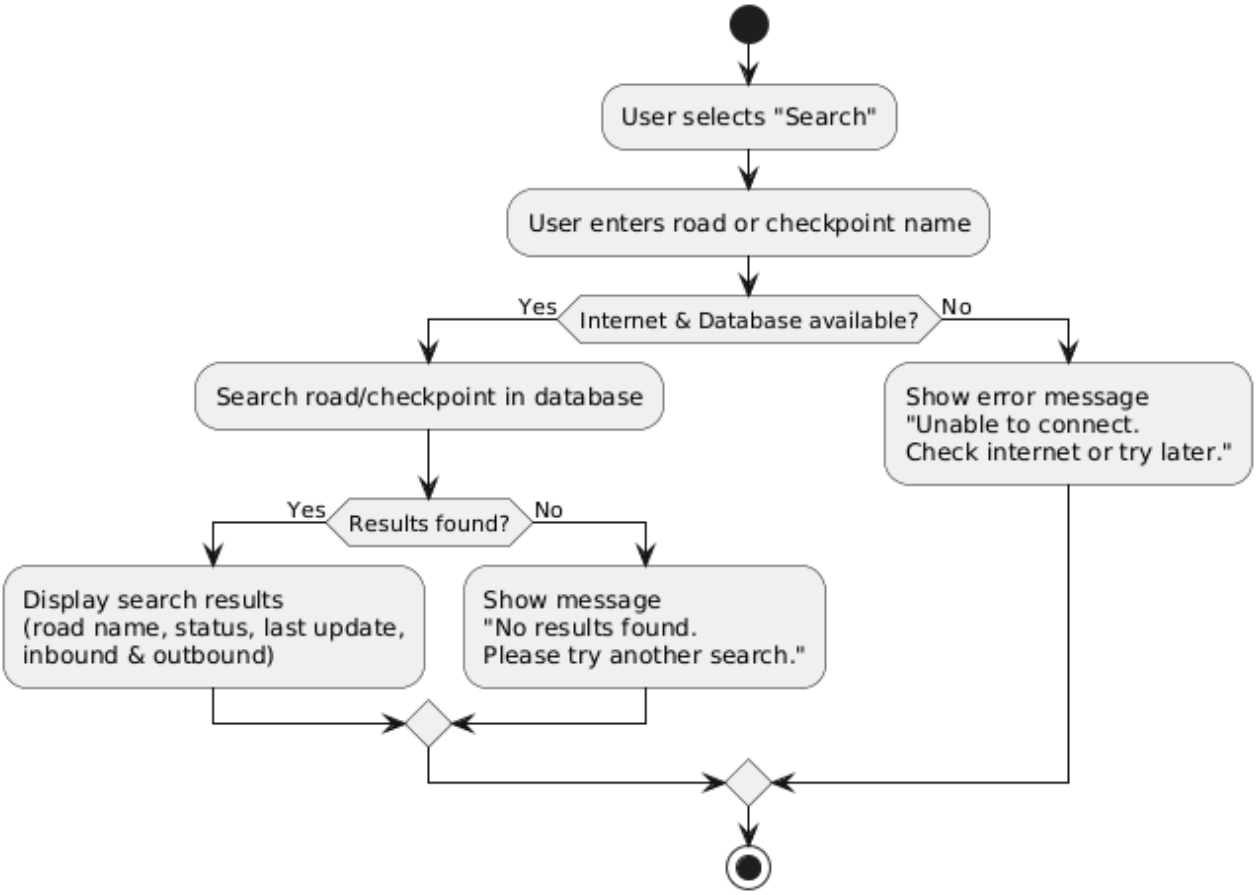

Figure 14:Search For Specific Road or Checkpoints Status Activity Diagram

3.3.4 State Chart Diagram

You can access the image by the following URL
https://drive.google.com/file/d/1ayFZRDu5ywdslrn3-TuDvMi6oF09G5d-/view?usp=sharing
o

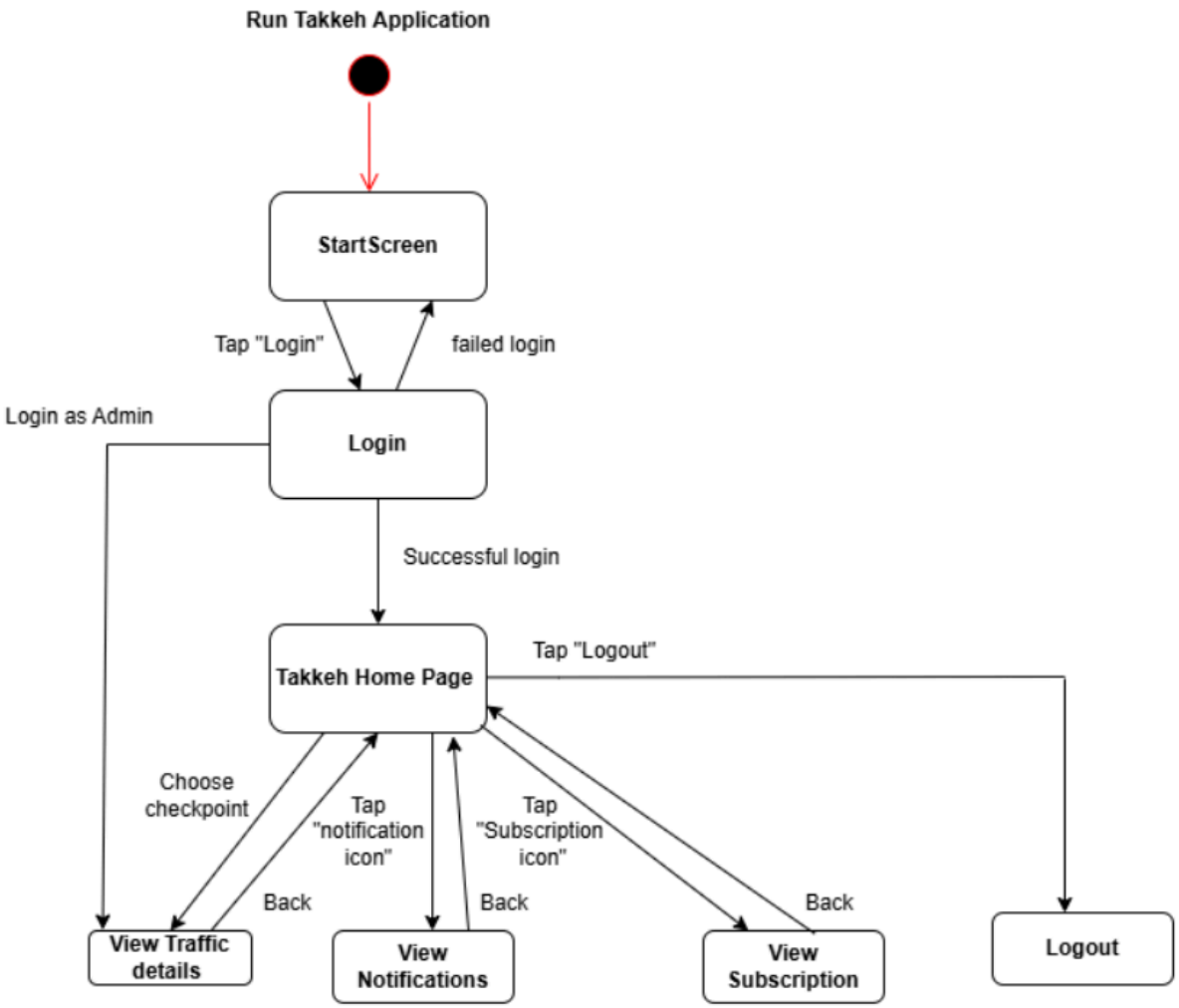

Figure 15:State Chart Diagram

3.4 System Architecture

3.4.1 Sub-System (descriptions of the sub-systems and their services)

1. User Management Subsystem

This module is responsible for user registration, login. It supports new users, registered users, and administrators. Users can securely log in through verified credentials against a database, which manages authentication. New users can create accounts using their email.

2. Incident Reporting Subsystem

Enables registered users to change road and checkpoint is status. Status changes require approximately five other user confirmations Within a time period of 30 minutes or less before being automatically implemented or reviewed by an admin.

3. Map Subsystem

This module provides a map indicating the user's relative position as obtained via GPS, as well as the location of known checkpoints. the user's location is updated via the device's GPS sensor.

4.Routing & Notification Sub-System

Users can select a city as their destination by activating a subscription there, and manually define their planned route by subscribing to the required checkpoints. The system proactively tracks the selected checkpoints and sends instant alerts if any are closed, congested, or reopened during the journey, ensuring users remain informed throughout the trip without the need for constant monitoring while driving.

5. Admin Dashboard Subsystem

This system is restricted to administrators only. It ensures complete system control and includes features such as user management, reviewing user suggestions, checking checkpoints, entering or modifying route data, administrator oversight of system logs, and monitoring user activity.

6.(NLP) Subsystem

This module focuses on the Telegram groups / supergroups traffic data using NLP, processes the unstructured data, captures relevant data, and stores them as clean, organized, and consistent reports in a database. The captured data which includes locations, status,direction and timestamp are standardized prior to storing the database.

7.Database Subsystem

This subsystem acts as the backend for persistence of data. The subsystem uses Realtime Database and Cloud Storage to save all user data, checkpoints, subscription, admin information, NLP processing and notification logs. This subsystem guarantees rapid access and synchronization of the data between devices.

3.4.2 NLP Processing Sub-System

The NLP Processing Sub-System represents a central component of the Takkah system architecture, specifically designed to address the challenge of analyzing traffic conditions and checkpoint information from unstructured Telegram messages. The main goal of this sub-system is to convert raw, unstructured text into structured traffic data, enabling the application to provide accurate real-time insights about road conditions.

By employing rule-based NLP techniques, the system achieves high accuracy, fast processing, and transparency, which are essential for a safety-critical application like Takkah[15]. Each incoming message passes sequentially through a series of stages: Text Normalization → Location Extraction → Direction Identification → Traffic Classification, ensuring a structured and systematic analysis pipeline.

3.4.2.1 Text Normalization

Text normalization is the first step in processing messages. Its purpose is to minimize linguistic variability and remove noise that could interfere with subsequent analysis stages. Telegram messages often contain irregular punctuation, emojis, repeated letters, and multiple forms of Arabic characters, which can prevent accurate recognition of locations and traffic keywords [17].

This stage includes:

- Transforming text into a standardized form.
- Simplifying punctuation, emojis, and extra symbols.
- Normalizing different forms of Arabic letters (e.g., converting variant forms of 'ا' to a single standard form).
- Removing elongations and repeated characters.

Scientific rationale: By standardizing the text, all rule-based matching operations in later stages can operate on consistent and noise-free input, improving detection accuracy for both location and traffic keywords.

3.4.2.2 Gazetteer-Based Location Extraction

Location extraction is performed using a gazetteer-based approach, a type of rule-based Named Entity Recognition (NER)[16]. In this context, a gazetteer is a predefined dictionary of geographic names that are important for the system, such as Palestinian towns, cities, and

frequently mentioned road areas.

Algorithm workflow:

1. The normalized message is tokenized into individual words and multiword phrases.
2. Each token or phrase is compared against the gazetteer.
3. When a match is found, the system identifies the location by name and records it as a structured data point.

Scientific rationale:

- The geographic domain is limited and precise, which allows rule-based matching to achieve high accuracy without machine learning.
- This approach ensures that locations are exactly detected, even in informal or abbreviated Telegram messages, which is critical for analyzing traffic patterns reliably.

Practical impact for Takkah: By extracting exact locations, the system can identify road segments and checkpoints mentioned in messages, enabling users to know where traffic disruptions are occurring.

3.4.2.3 Direction Detection

After extracting location entities, the system performs direction detection to determine whether the reported traffic condition applies to inbound traffic, outbound traffic, or both directions. This step is implemented using a rule-based approach that relies on predefined directional keywords and common linguistic patterns found in Arabic traffic-related messages.

Algorithm workflow:

1. The normalized message is scanned for directional keywords and expressions that indicate movement, such as terms referring to incoming, outgoing, toward, or away from a specific location.
2. When directional indicators are detected, the system maps them to a standardized direction label (e.g., inbound or outbound) and associates them with the previously extracted location.
3. If no explicit directional information is found, the system assumes that the traffic condition applies to both directions by default.

Scientific rationale:

Traffic-related messages typically use a limited and repetitive set of directional expressions. Therefore, a rule-based approach is well-suited for direction detection, providing high reliability while maintaining transparency, interpretability, and low computational complexity compared to machine learning-based models.

Practical impact for Takkah: Direction detection enables the system to distinguish traffic conditions by movement direction on the same road segment. This improves the accuracy of map visualization and allows users to receive more relevant and precise traffic notifications based on their actual travel direction.

3.4.2.4 Traffic Status Classification (Weighted Pattern Matching)

After detecting locations, the system applies weighted pattern matching to classify the type and severity of traffic conditions.

- Predefined traffic keywords such as *'open,' 'accident,' 'checkpoint,' 'traffic,' 'closed'* are assigned weights according to severity.
- The system calculates a cumulative score for the message based on matched keywords.
- The resulting score determines the traffic condition classification, ranging from smooth traffic to heavy congestion or blockage.

Scientific rationale:

- Weighted pattern matching allows quantitative assessment of message content without requiring machine learning models [18].
- The approach is transparent, enabling system developers to explain why a message was classified in a certain way.

Practical impact for Takkah: Users receive actionable information about road conditions and checkpoint presence, allowing real-time route planning and awareness.

3.4.3 Software Architecture

You can access the image by the following URL
https://drive.google.com/file/d/1wfaF5poSIktUa6qpCvtq-uRR_7iFQio0/view?usp=sharing

This architecture demonstrates how the layers of UI, logic (middle), and infrastructure interact. Users access registration, feeds, and maps while core components manage a user, updates on checkpoints, and notifications for the user. A GPS service and cloud database support technology stack allows real-time tracking and data storage.

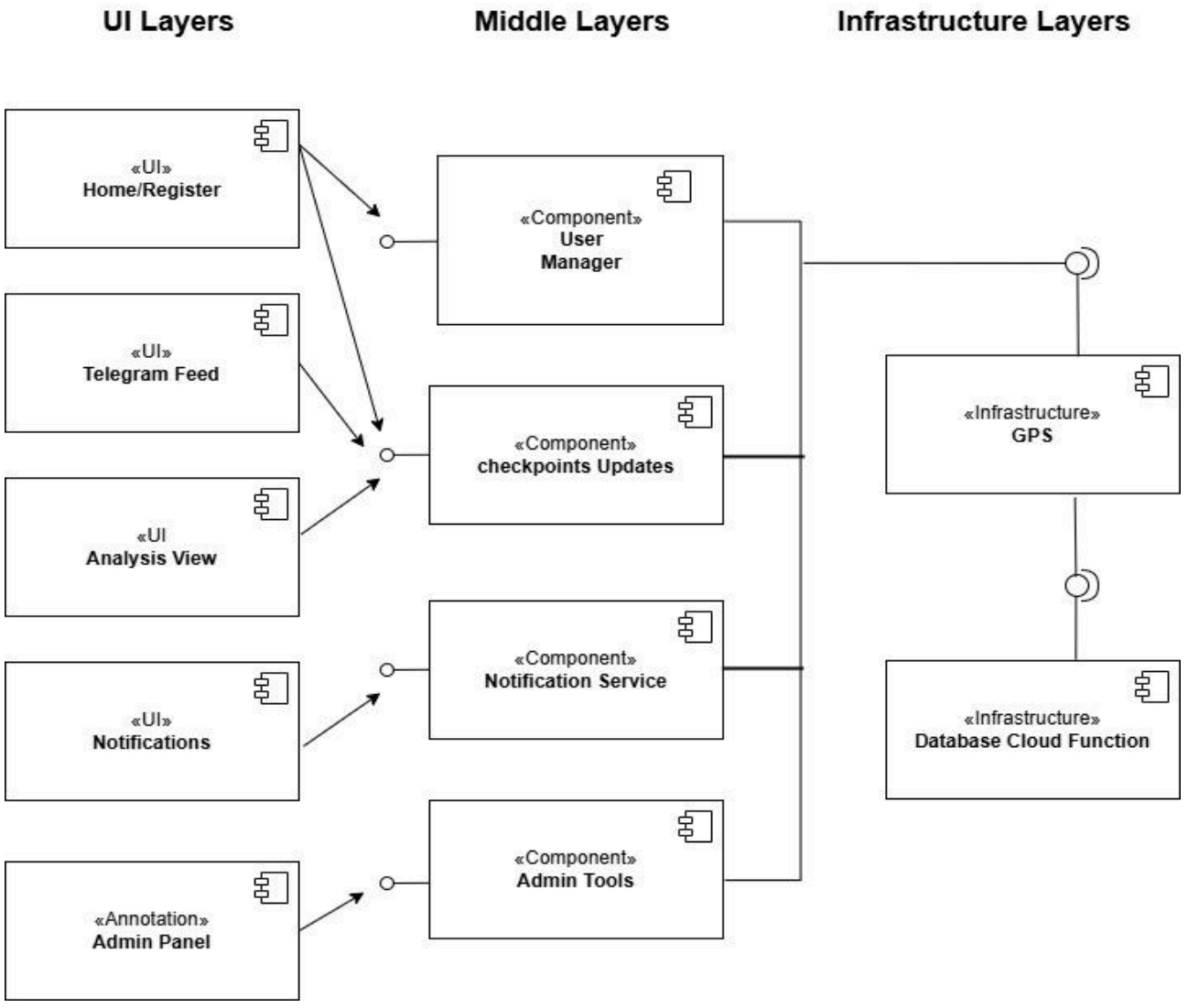

Figure 16: Architecture Diagram

3.4.4 Deployment diagram

This deployment diagram shows the physical structure of the Takkeh system. It illustrates how the mobile app on the client side connects with the backend server, which then communicates with a different database server. This distinct separation promotes security, scalability, and ease of maintenance.

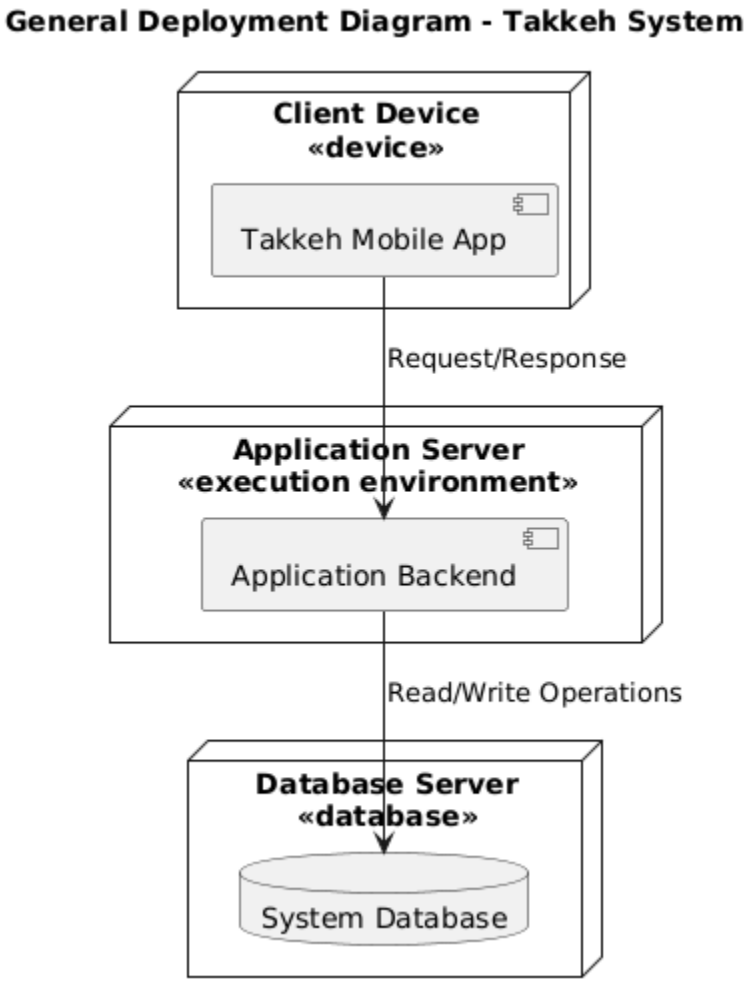

Figure 17: Deployment Diagram

3.4.5 Component diagram

You can access the image by the following URL

https://drive.google.com/file/d/1oqBgnL6vuI_ay2JpNXTjTyefjWUO0YJR/view?usp=sharing

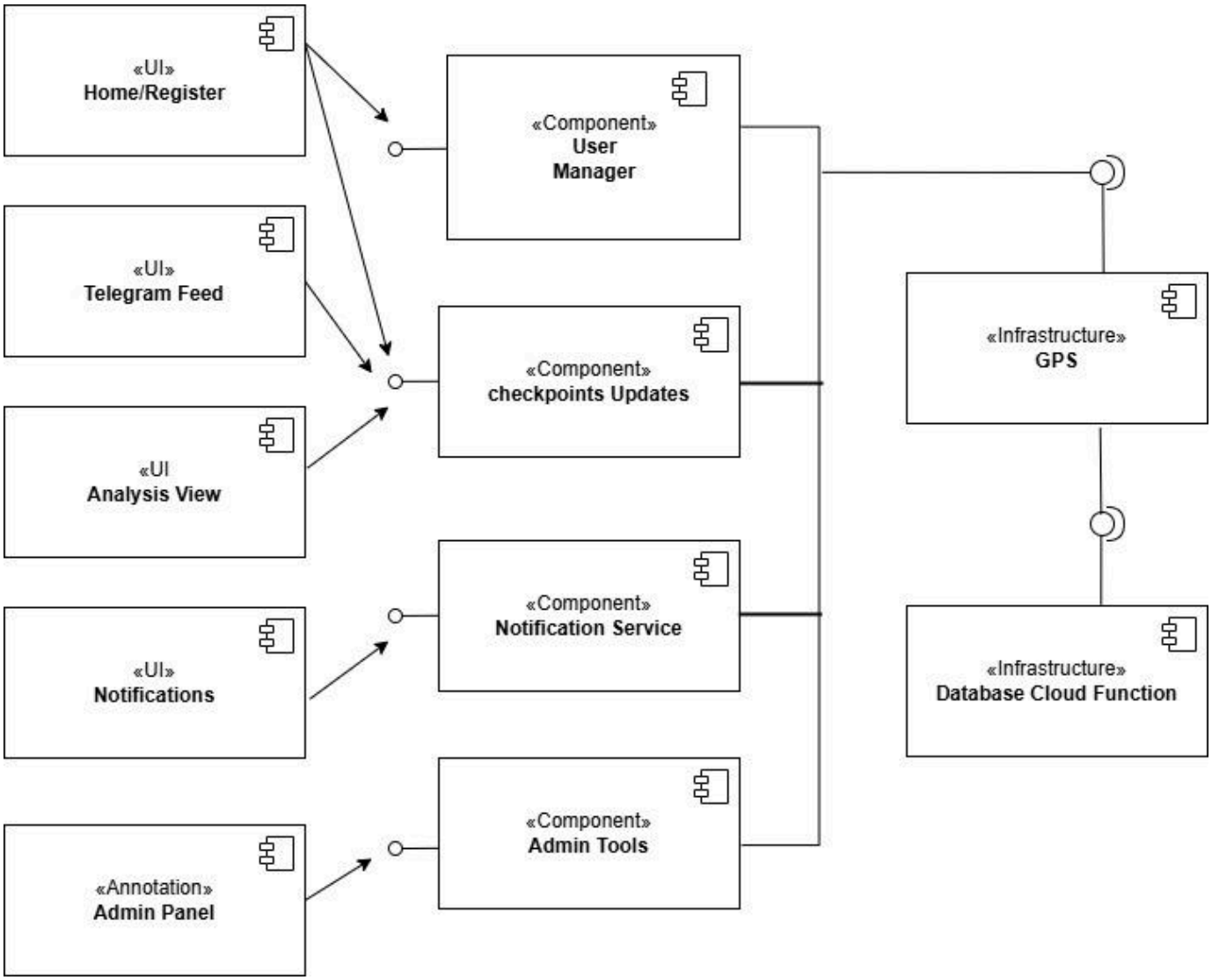

Figure 18: Component Diagram

3.5 Data Management and ERD

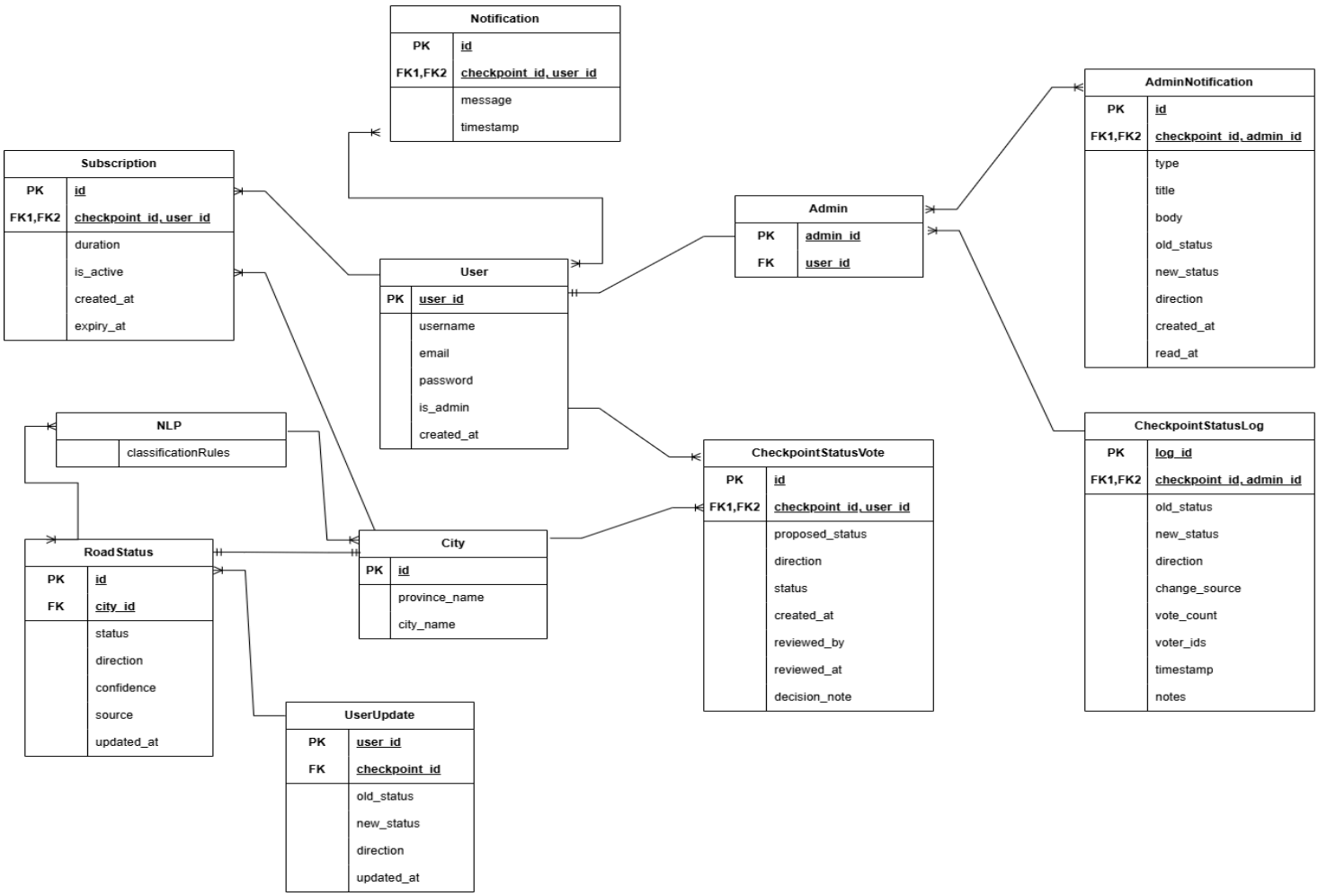

Figure 19: ERD Diagram

Chapter 4: Implementation

4.1: User

4.1.1 Home Page

This interface is the main home page for the user. It contains a search bar to search for a city or checkpoint, with filtering options to refine the results.

The page also includes a map icon to open the map view. The bottom navigation bar provides access to My Account, My Subscriptions, and Notifications.

Figure 20: Home Page

4.1.2 Filter Icon

This interface lets users filter checkpoints by direction (inbound, outbound, or both) and current road status (open, closed, checkpoint, accident, or traffic congestion). Users can apply or clear the selected filters whenever they want.

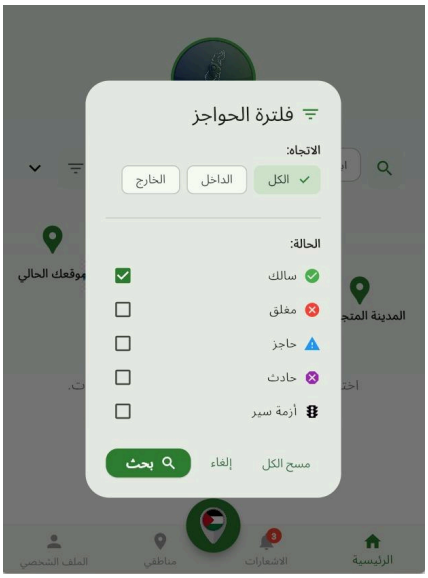

Figure 21: Filter Icon

4.1.3 Notification Screen

This interface displays notifications related to checkpoint status updates, including the affected direction and update time. It also shows admin promotion requests sent to the user, allowing the user to accept or decline the request to become an admin.

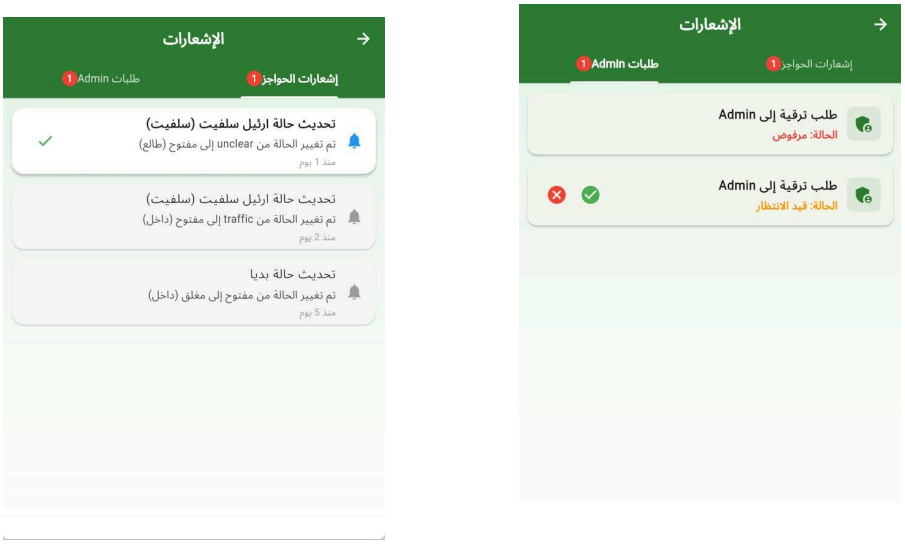

Figure 20: Notification Screen

4.1.4 Map Screen

This interface shows the user's active checkpoint subscriptions and remaining subscription time.

Figure 21: Map Screen

4.1.5 Subscriptions Screen

This interface shows the user's active checkpoint subscriptions and remaining subscription time.

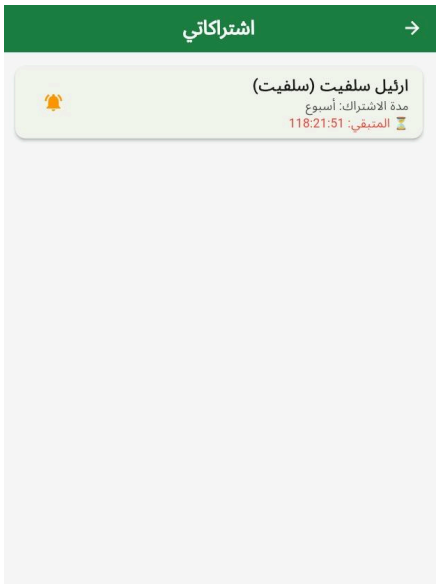

Figure 22: Subscriptions Screen

4.1.6 Checkpoint details with subscription icon and the ability to change road status page

This interface shows the checkpoint status and allows users to suggest changes and subscribe to the checkpoint.

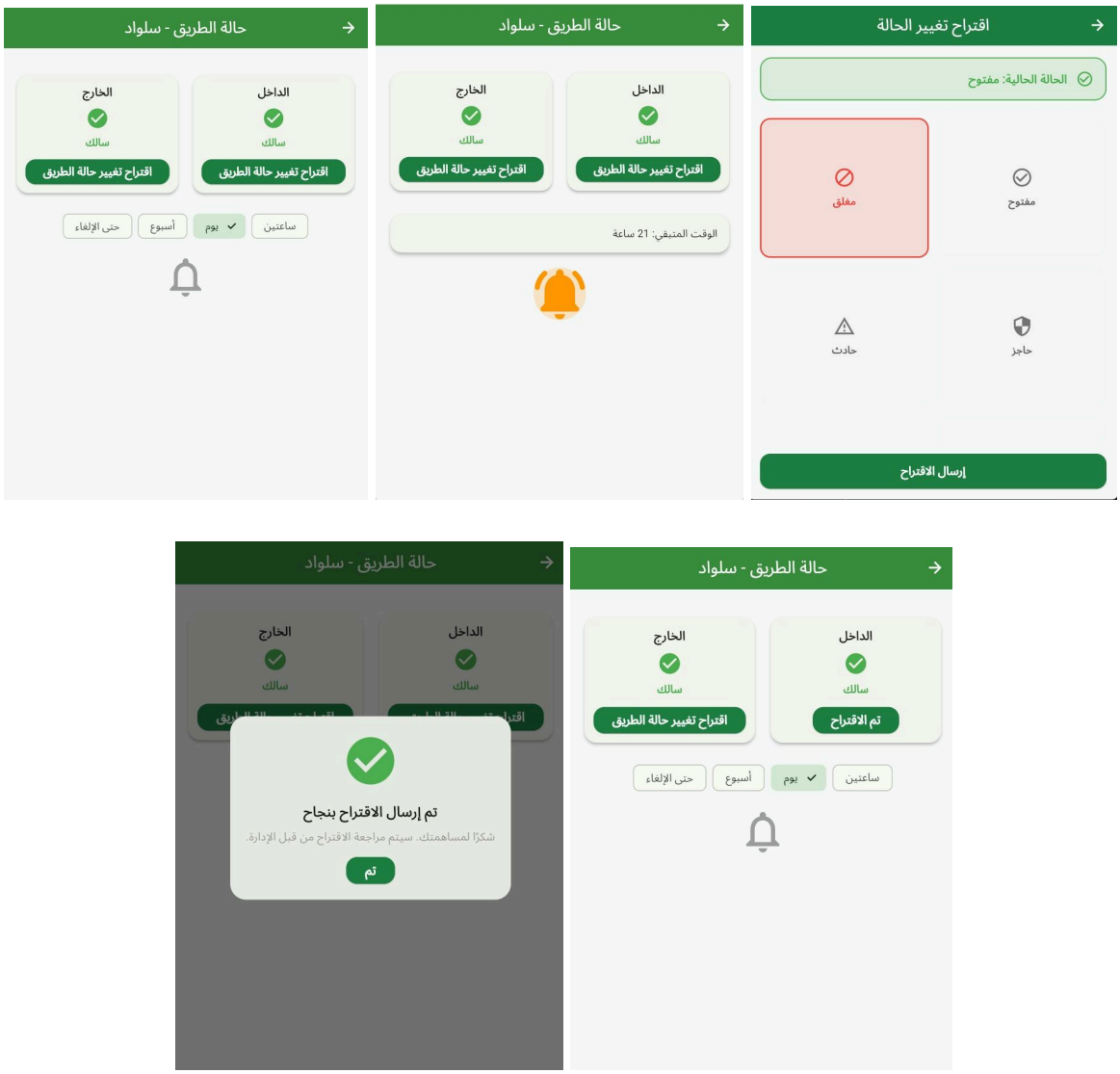

Figure 23: Checkpoint details with subscription icon and the ability to change road status page

4.2: Admin

4.2.1 Admin Dashboard

This dashboard provides admins with an overview of system statistics, recent updates, user activity, and NLP performance metrics.

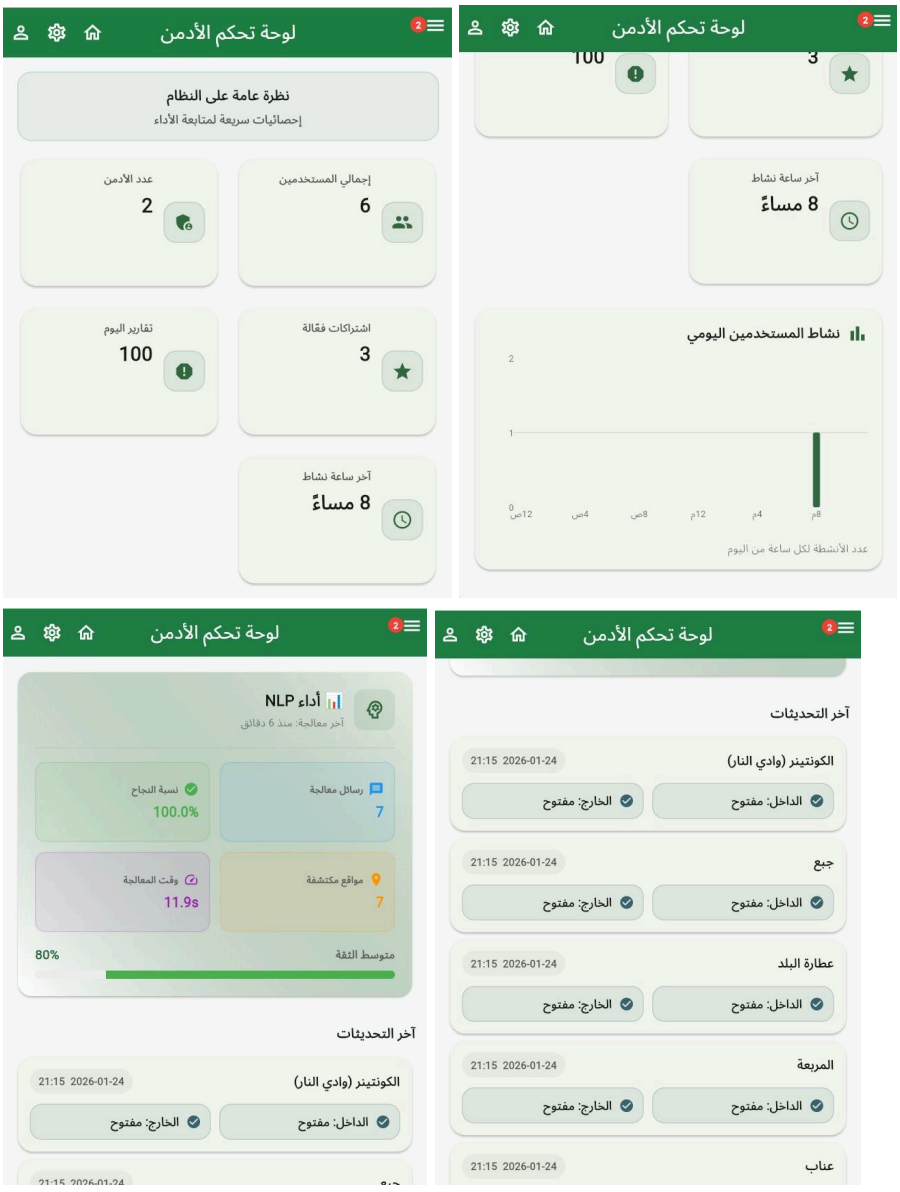

Figure 24: Admin Dashboard

4.2.2 Users Management Page

This interface enables admins to manage users, filter them by role, and view their current status.

Figure 25: Users Management Page

4.2.3 Checkpoints Management Page

This interface enables admins to manage checkpoint statuses and update inbound and outbound directions.

Figure 26: Checkpoints Management Page

4.2.4 Users Activities Page

This interface shows a history of checkpoint status changes, which includes modifications made by administrators or through user votes. It presents the old and new status, the direction impacted, the origin of the change, and the time it happened.

Figure 27: Users Activities Page

4.2.5 Users Subscriptions Page

This interface enables admins to monitor active user subscriptions and evaluate user engagement with the subscription feature.

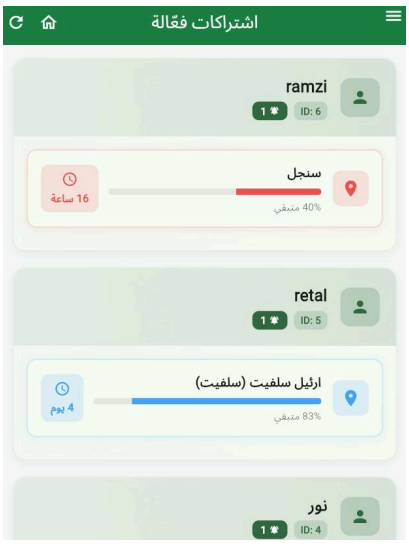

Figure 28: Users Subscriptions Page

4.2.6 Admin Notification Screen

This interface shows admin notifications for road status suggestions and automatic updates, allowing approval or rejection.

Figure 29: Admin Notification Screen

Chapter 5: Testing

5.1 Feature 1: Map - User Usage

5.1.1 : Test Case 1

Happy Flow

TEST CASE TITLE	TEST DATE
Open Map Page from Home (User)	19/1/2026

TEST SCENARIO	TEST DESIGNED BY
User navigates from Home to Map page	Nour Manasrah

TEST DESCRIPTION	PRE-Conditions
Verify that a logged-in user can open the Map page from the Home screen and the map loads successfully.	1. User is registered and logged in. 2. Device has active internet connection. 3. Location permission is allowed (if the app requires it to render the map). 4. At least one map provider/service is configured and reachable. 5. App opens to Home screen (default landing after login).

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	Launch the app	App opens successfully and shows Home screen.	App opened successfully and Home screen was displayed	PASS	Application launched without crashes or delays
2	Verify user is logged in such as (user profile shows the username and email)	User is recognized as logged in; no login screen is shown.	User session was active and user information was visible on the Home screen. Login screen was not shown	PASS	

3	Locate the “Map” entry point on Home (button/tab/icon).	“Map” entry point is visible and enabled	“Map” entry point was visible and enabled on the Home screen.	PASS	
4	Tap the “Map” entry point.	App navigates to the Map page without errors.	Application navigated to the Map page successfully without errors.	PASS	
5	Wait for the Map page to load	Map loads immediately without displaying a loading indicator.	Map loaded immediately without showing a loading indicator.	PASS	
6	Verify that the map tiles/graphics are rendered	Map is displayed (tiles load and no blank/gray screen).	Map tiles and markers were rendered correctly. Areas with no analyzed road data were displayed in gray as expected	PASS	Gray markers indicate areas with no available road status data.
7	Verify that basic map interactions work (pan/zoom)	User can pan and zoom the map smoothly	User was able to pan and zoom the map smoothly without lag	PASS	Basic map interactions worked as expected

Table 3: Test Case 1 For Map - User Usage

5.1.2 :Test Case 2

Happy Flow

TEST CASE TITLE	TEST DATE
View Checkpoint Details and Road Status from Map	19/1/2026

TEST SCENARIO	TEST DESIGNED BY
User taps a checkpoint marker on the map to view detailed road status information	Nour Manasrah

TEST DESCRIPTION	PRE-Conditions
Verify that checkpoint markers are displayed on the map and that tapping a checkpoint marker opens a details view showing accurate road status information for the selected checkpoint.	1. User is registered and logged in 2. Internet connection is available 3. Map page is accessible 4. Backend contains checkpoint records (at least 3) with status + last updated info 5. Map loads successfully

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	Launch the app	App opens successfully and shows Home screen	App opened successfully and Home screen was displayed	PASS	Application launched without crashes or delays
2	Verify that the user is logged in (user profile information is visible)	User is recognized as logged in; no login screen is shown	User session was active and user information was visible on the Home screen. Login screen was not shown	PASS	
3	Locate the “Map” entry point on Home (button/tab/icon)	“Map” entry point is visible and enabled	“Map” entry point was visible and enabled on the Home screen	PASS	

4	Tap the “Map” entry point	App navigates to the Map page without errors	Application navigated to the Map page successfully without errors	PASS	
5	Wait for the Map page to load	Map loads immediately without displaying a loading indicator	Map loaded immediately without showing a loading indicator	PASS	
6	Identify a visible checkpoint marker on the map	At least one checkpoint marker is visible and tappable on the map	Checkpoint markers were visible and tappable. Markers with no analyzed road data were displayed in gray as expected	PASS	Gray markers indicate checkpoints with unknown or unprocessed road status
7	Tap the checkpoint marker	A details view opens for the selected checkpoint (card/popup/bottom sheet/page)	Checkpoint details view opened successfully for the selected marker	PASS	
8	Verify checkpoint details displayed	Details view shows checkpoint information such as: Checkpoint name, Road status (Open / Closed / Unknown “Gray”),Last updated time (date and time)	Checkpoint details were displayed correctly, including name, road status, and last updated time	PASS	
9	Validate that the details belong to the selected marker	The displayed checkpoint name/location matches the tapped marker (no mismatch/wrong checkpoint)	Displayed checkpoint details matched the selected marker correctly	PASS	
10	Close the details view / navigate back to the map	User returns to the map without errors; markers remain visible and map remains responsive	User returned to the map successfully. All checkpoint markers remained visible and the map was responsive	PASS	

Table 4: Test Case 2 For Map - User Usage

5.1.3 : Test Case 3

Negative Flow

TEST CASE TITLE	TEST DATE
New checkpoint updates are not loaded when internet is unavailable, while previously cached data remains visible	19/1/2026

TEST SCENARIO	TEST DESIGNED BY
User opens the Map page with no internet connection and views previously cached checkpoint data, while new updates fail to load	Nour Manasrah

TEST DESCRIPTION	PRE-Conditions
Verify that when the internet connection is unavailable, the application displays previously cached checkpoint data on the map, prevents loading new updates, and informs the user with a clear message without crashing or displaying incorrect data	1. User is registered and logged in 2. Device had internet connection previously and map data was loaded 3. Checkpoint markers are already visible on the map 4. Internet connection is disabled after initial load

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	Launch the application while internet is enabled	App opens successfully and map data loads normally.	Application opened successfully and map data was loaded	PASS	Application launched without crashes or delays
2	Verify user is logged in such as (user profile shows the username and email)	User is recognized as logged in; no login screen is shown.	User session was active and user information was visible on the Home screen. Login screen was not shown	PASS	
3	Locate the “Map” entry point on Home (button/tab/icon).	“Map” entry point is visible and enabled	“Map” entry point was visible and enabled on the Home screen.	PASS	
4	Tap the “Map” entry point.	App navigates to the Map page without errors.	Application navigated to the Map page	PASS	

			successfully without errors.		
5	Wait for the Map page to load	Map loads immediately without displaying a loading indicator.	Map loaded immediately without showing a loading indicator.	PASS	
6	Verify that the map tiles/graphics are rendered	Map is displayed (tiles load and no blank/gray screen).	Map tiles and markers were rendered correctly. Areas with no analyzed road data were displayed in gray as expected	PASS	Gray markers indicate areas with no available road status data.
7	Disable internet connection (turn off Wi-Fi / mobile data)	Device is offline.	Internet connection was disabled successfully	PASS	
8	Observe the map after internet is disabled	Previously loaded checkpoint markers remain visible on the map	Cached checkpoint markers remained visible on the map	PASS	
9	Wait for new checkpoint updates or attempt to refresh the map	New updates are not loaded and a clear message is shown indicating updates cannot be fetched	New updates were not loaded and a message was displayed: "تعذّر تحميل التحديثات الآن"	PASS	
10	Verify application stability while offline	Application remains stable with no crashes or incorrect data shown	Application remained stable and responsive with no errors	PASS	

Table 5: Test Case 3 For Map - User Usage

5.2 Feature 2: Notifications

5.2.1: Test Case 1

Happy Flow

TEST CASE TITLE	TEST DATE
Verify User Can Successfully View Checkpoint Notifications List	21/1/2026

TEST SCENARIO	TEST DESIGNED BY
User navigates to notifications page to view checkpoint status updates	Roaa Ali Ahmad

TEST DESCRIPTION	PRE-Conditions
This test case verifies that when a logged-in user navigates to the notifications page and selects the checkpoint notifications tab, all checkpoint notifications are fetched from the database and displayed correctly in the list view.	1. User is registered and logged in 2. Notifications permission is granted on the device 3. Device has active internet connection 4. Checkpoint details page is accessible 5. Notification service is available

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	Launch the app.	App opens successfully and shows Home screen.	App opened successfully and Home screen was displayed	PASS	Application launched without crashes or delays
2	Verify user is logged in such as (user profile shows the username and email)	User is recognized as logged in; no login screen is shown.	User session was active and user information was visible on the Home screen. Login screen was not shown	PASS	
3	Navigate to User Notifications Page	User Notifications Page opens and displays the	Notifications page opened correctly showing the title	PASS	

		page title "الإشعارات" in the app bar with two tabs visible below the title	and two tabs		
4	Verify default tab selection	The first tab "إشعارات الحواجز" (Checkpoint Notifications) is selected by default	"إشعارات الحواجز" tab was selected automatically	PASS	
5	Verify notifications list loads automatically	Loading completes within 3 seconds and a list of notifications appears on screen	List loaded in 2 seconds with all notifications displayed	PASS	
6	Verify notification card structure for unread notification	- Card has white background color - Card has elevation shadow - Blue notification bell icon is displayed on the leading side - Title text is displayed in bold font weight - Body/description text is displayed below the title - Timestamp is displayed in small grey text showing relative time (like: "منذ 5 دقيقة") - Green check icon button is displayed on the trailing side	All expected UI elements displayed correctly with proper styling	PASS	
7	Verify notification card structure for read notification	- Card has grey background color - Card has minimal elevation shadow - Grey notification bell icon is displayed on the	Read notification displayed with grey background and normal styling as expected	PASS	

		leading side - Title text is displayed in normal font weight (not bold) - Body text and timestamp are displayed normally - No check icon button on the trailing side			
8	Verify notifications are ordered correctly	Notifications are sorted by creation date with the most recent notification at the top of the list	Notifications displayed in correct chronological order	PASS	
9	Verify notification content accuracy	- Title contains meaningful checkpoint update information - Body contains detailed description of the status change - All text is in Arabic language - Timestamp shows correct relative time	All notification content accurate, in Arabic, with correct relative timestamps	PASS	

Table 6: Test Case 1 For Notifications

5.2.2: Test Case 2

Happy Flow

TEST CASE TITLE	TEST DATE
Verify User Can Successfully Accept Admin Promotion Request	21/1/2026

TEST SCENARIO	TEST DESIGNED BY
User receives an admin promotion request notification and accepts it to become an admin	Roaa Ali Ahmad

TEST DESCRIPTION	PRE-Conditions
This test case verifies that when a user navigates to admin requests notifications tab and taps the accept button on a pending admin promotion request, the request status is updated to accepted, user is added to admins table, and UI displays success confirmation.	1. User is logged in 2. User has at least one pending admin promotion request 3. User is not currently an admin 4. Internet connection is active

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	Launch the app.	App opens successfully and shows Home screen.	App opened successfully and Home screen was displayed	PASS	Application launched without crashes or delays
2	Verify user is logged in such as (user profile shows the username and email)	User is recognized as logged in; no login screen is shown.	User session was active and user information was visible on the Home screen. Login screen was not shown	PASS	
3	Navigate to User Notifications Page	Notifications page opens with two tabs visible	Notifications page opened successfully showing both tabs	PASS	
4	Tap on "طلبات Admin" tab	Admin requests tab selected and list of admin requests displayed	"طلبات Admin" tab selected, showing 1 pending request	PASS	

5	Locate a pending admin request notification	Notification card shows title "طلب ترقية إلى Admin" with status "pending" in orange color	Found pending request with orange status indicator	PASS	
6	Verify action buttons are present	Green check icon button and red close icon button visible on the trailing side	Both accept (green check) and reject (red X) buttons displayed	PASS	
7	Tap on the green check icon button (Accept)	Button responds with visual feedback	Button pressed with ripple effect animation	PASS	
8	Observe immediate UI feedback	Green success message bar appears in center: "تم قبول الطلب! ستصبح Admin الآن."	Success message displayed in green container with correct text	PASS	
9	Verify request status update	Request status changes from "pending" to "accepted" with green color	Status updated to "accepted" and displayed in green color	PASS	
10	Check for admin star icon	Gold/amber star icon appears on trailing side of the accepted request	Gold star icon appeared indicating admin status	PASS	
11	Verify action buttons are removed	Accept and reject buttons no longer visible for accepted request	Action buttons removed after acceptance	PASS	

Table 7: Test Case 2 For Notifications

5.2.3:Test Case 3

Happy Flow

TEST CASE TITLE	TEST DATE
Verify Admin Receives Notification When User Suggests Checkpoint Status Change and Can Approve It	21/1/2026

TEST SCENARIO	TEST DESIGNED BY
User submits a checkpoint status change suggestion, admin receives notification and approves the suggestion	Roaa Ali Ahmad

TEST DESCRIPTION	PRE-Conditions
This test case verifies that when a regular user proposes a checkpoint status change, the admin receives a notification in real-time, can view the suggestion details, and successfully approve or reject the suggestion.	1. Regular user is logged in (not an admin) 2. Admin user is logged in on a separate session/device 3. At least one checkpoint exists in the system 4. User has not voted on this checkpoint within the last 30 minutes (no cooldown active) 5. Internet connection is active 6. Admin is on the Admin Notifications Page

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	Launch the app.	App opens successfully and shows Home screen.	App opened successfully and Home screen was displayed	PASS	Application launched without crashes or delays
2	Verify user is logged in such as (user profile shows the username and email)	User is recognized as logged in; no login screen is shown.	User session was active and user information was visible on the Home screen. Login screen was not shown	PASS	
3	User navigates to a checkpoint details page	Checkpoint page displays with current status and "اقتراح تغيير الحالة" button visible	Checkpoint page loaded showing for instance status "مفتوح" with suggestion button	PASS	

4	User taps on "اقتراح تغيير الحالة" button	Propose Checkpoint Status Page opens showing current status and status options grid	Proposal page opened displaying current status card and 5 status options	PASS	
5	User views current checkpoint status	Current status displayed in colored card at top of screen	Current status "مفتوح" displayed in green card with check icon	PASS	
6	User selects a new status from the grid (like "حاجز")	Selected status card highlights with colored border and background	"حاجز" option selected with blue border and light blue background	PASS	
7	User taps "إرسال الاقتراح" button at bottom	Confirmation dialog appears asking "من مفتوح إلى حاجز ؟" with cancel and confirm buttons	Confirmation dialog displayed with correct old and new status labels	PASS	
8	User taps "تأكيد" button in confirmation dialog	Loading indicator appears, request sent to server	Loading spinner displayed on submit button during API call	PASS	
9	Server processes suggestion and creates admin notification	Server returns success response (status 200) with vote recorded	Server responded successfully, vote saved to database	PASS	
10	User sees success dialog	Success dialog appears with message "وصل اقتراحك ✅"	Success dialog displayed with confirmation message	PASS	
11	Admin page automatically updates (real-time stream)	New notification appears at top of admin notifications list without manual refresh	New notification appeared instantly in admin's notifications list	PASS	
12	Admin views notification details	Notification shows: title "اقتراح حالة طريق جديدة"، username chip, checkpoint name chip, direction chip, old→new status in body	All notification details displayed correctly with chips showing user "أحمد", checkpoint "قلنديا", direction "داخل"	PASS	
13	Admin verifies notification is marked as unread	Notification has primary color avatar, bold title, and colored timestamp	Notification displayed with blue avatar, bold title, and blue timestamp	PASS	
14	Admin views action buttons on notification	Two buttons visible: "قبول" (green with check icon) and "رفض" (outlined with X icon)	Both accept and reject buttons displayed under notification body	PASS	

15	Admin taps "قبول" button to approve suggestion	Loading indicator appears on button, approve request sent to server	Loading spinner showed on accept button during processing	PASS	
16	Server processes admin approval	Suggestion approved, checkpoint status updated in database, notification marked as handled	Server returned success, checkpoint status changed from "مفتوح" to "حاجز"	PASS	
17	Admin sees success confirmation	Green snackbar appears at bottom: "تم القبول ✅"	Success snackbar displayed with confirmation message	PASS	
18	Admin verifies action buttons disappear	Accept and reject buttons removed from notification after approval	Action buttons removed, notification shows as processed	PASS	
19	Verify checkpoint status updated in system	Navigate to checkpoint page and confirm status changed to approved value	Checkpoint page shows updated status "حاجز" instead of "مفتوح"	PASS	

Table 8: Test Case 3 For Notifications

5.3 Feature 3: Reports & Update Road Status

5.3.1: Test Case 1

Happy Flow

TEST CASE TITLE	TEST DATE
Verify System Automatically Changes Checkpoint Status After 5 Different Users Vote	21/1/2026

TEST SCENARIO	TEST DESIGNED BY
Five different users vote for the same new status on the same checkpoint and direction within 30 minutes	Enas Hamayel

TEST DESCRIPTION	PRE-Conditions
This test case verifies that when 5 unique users vote for the same checkpoint status change (same direction) within the cooldown period, the system automatically approves the change, updates the database, logs the change, and notifies subscribed users.	1. At least 5 different user accounts are registered and logged in 2. A checkpoint exists with current status like "مفتوح" for inbound direction) 3. No user has voted on this checkpoint/direction within the last 30 minutes

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	User 1 logs in, searches for checkpoint "قلنديا", opens checkpoint details screen	Checkpoint details page is displayed successfully	Page displayed	PASS	
2	User 1 selects Inbound (الداخل) direction and taps "اقتراح تغيير حالة الطريق"	Status selection screen is displayed	Screen displayed"	PASS	
3	User 1 selects "حاجز" and submits the proposal	Confirmation message appears ("شكراً لمساهمتك") and cooldown timer starts	Message shown, cooldown started	PASS	Cooldown = 30 min

4	Users 2–5 repeat steps 1–3 within 30 minutes, selecting the same checkpoint, direction, and status "حاجز"	Each proposal is accepted and recorded; system tracks vote count internally	All proposals accepted	PASS	Unique users only
5	Fifth proposal is submitted	System automatically approves the status change	Status auto-approved	PASS	No admin action
6	System updates checkpoint status for Inbound direction to "حاجز"	Checkpoint status is updated in database	Status updated	PASS	
7	System logs the change	Log entry created with source = "voting" and vote count = 5	Log exists	PASS	
8	System sends notifications	Subscribed users receive push notifications	Notifications received	PASS	
9	User app updates UI	Checkpoint shows "حاجز" immediately without manual refresh	Status updated instantly	PASS	Real-time update
10	Admin panel reflects the change	Admin sees automatic status change in admin notification screen with source "voting"	Change visible	PASS	

Table 9 : Test Case 1 For Reports & Update Road Status

5.3.2:Test Case 2

Negative Flow

TEST CASE TITLE	TEST DATE
Verify User Cannot Submit Multiple Votes Within 30-Minute Cooldown Period	21/1/2026

TEST SCENARIO	TEST DESIGNED BY
User attempts to vote on the same checkpoint and direction before the 30-minute cooldown expires	Enas Hamayel

TEST DESCRIPTION	PRE-Conditions
This test case verifies that the system enforces a 30-minute cooldown period per user per checkpoint per direction, preventing vote spam and ensuring vote integrity.	1. User is logged in 2. User has already voted on a specific checkpoint and direction 3. Less than 30 minutes have passed since the last vote 4. The vote timestamp is stored in `checkpoint_status_votes` table

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	User opens the app and logs in successfully	App opens to home page showing provinces list	App launched, home page displayed with all provinces	PASS	
2	User taps on checkpoint "قلنديا" for example	Checkpoint details page opens showing current status for both directions (inbound: "مفتوح", outbound: "مفتوح")	Checkpoint page loaded displaying both statuses with details	PASS	
3	User taps "اقتراح تغيير الحالة" button	Proposal page opens showing current status card and grid of 5 status options	Proposal page opened showing current "مفتوح" status and status selection grid	PASS	

4	User selects "مغلق" status from the grid	Selected status highlights with colored border	"مغلق" option selected with red border and background	PASS	
5	User taps "إرسال الاقتراح" button at bottom	Confirmation dialog appears: "من مفتوح إلى مغلق ؟" with إلغاء and تأكيد buttons	Confirmation dialog displayed with correct status labels	PASS	
6	User taps "تأكيد" button	Loading indicator shows, vote request sent to server at 10:00 AM	Loading spinner displayed, API request sent successfully	PASS	
7	Server accepts vote and saves to database	Vote saved with timestamp, success dialog appears: "وصل اقتراحك ✅", 30-minute cooldown starts	Vote saved at 10:00 AM, success message shown, cooldown timer started	PASS	
8	User closes success dialog and navigates back to checkpoint page	Returns to checkpoint details page showing current status	Returned to قلنديا page successfully	PASS	
9	User waits 5 minutes then taps "اقتراح تغيير الحالة" again (at 10:05 AM)	Proposal page opens, cooldown timer may be visible	Proposal page opened, showing cooldown message or timer	PASS	
10	User selects any status and taps "إرسال الاقتراح"	System checks recent votes, calculates remaining time (25 min = 1500 sec), rejects vote	System queried database and calculated cooldown: 1500 seconds remaining	PASS	
11	Error dialog appears with cooldown message	Dialog shows: "لا يمكنك الاقتراح الآن، انتظر حتى انتهاء الوقت." with remaining time	Error message displayed with Arabic text indicating cooldown active	PASS	

Table 10 : Test Case 2 For Reports & Update Road Status

5.3.3: Test Case 3

Happy Flow

TEST CASE TITLE	TEST DATE
Verify Admin Can Manually Update Checkpoint Status and Changes Reflect in Real-Time	21/1/2026

TEST SCENARIO	TEST DESIGNED BY
Admin user manually changes checkpoint status from admin panel and the change reflects immediately across all user interfaces	Enas Hamayel

TEST DESCRIPTION	PRE-Conditions
This test case verifies that when an admin manually updates a checkpoint status, the change is saved to the database, logged properly, notifications are sent to subscribers, and the updated status appears immediately in the user app without requiring manual refresh.	1. Admin user is logged in to admin panel 2. Admin is on "ادارة الحواجز" page 3. At least one checkpoint exists with known current status 4. At least one regular user has the user app open viewing checkpoints 5. At least one user is subscribed to the checkpoint being updated

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS/ FAIL	ADDITIONAL NOTES
1	Admin logs in and navigates to "إدارة الحواجز" page	Admin checkpoints management page loads showing all checkpoints grouped by province	Page loaded showing checkpoints organized by محافظة	PASS	
2	Admin locates checkpoint "قلنديا" and clicks edit icon next to inbound status "مفتوح"	PopupMenu opens with 5 status options (مفتوح, مغلق, حاجز, حادث, أزمة سير	Found قلنديا with inbound "مفتوح" in green, popup menu opened successfully	PASS	
3	Admin selects "حاجز"	Confirmation dialog	Confirmation dialog	PASS	

	status and confirms in dialog	appears, admin clicks "تأكيد", loading indicator shows	displayed: "هل أنت متأكد من تغيير حالة 'قلنديا (داخل)' إلى 'حاجز'؟", confirmed successfully		
4	Backend processes update and saves to database	API request sent to `/admin/update-checkpoint-status`, `road_status` table updated, `inbound_status` changed from "open" to "checkpoint"	Request sent successfully, database updated: inbound_status="checkpoint"	PASS	
5	System logs change and sends notifications to subscribers	Log created in `checkpoint_status_logs` with admin info (admin_id=1, admin_name="أحمد", change_source="admin"). FCM notifications sent to 2 subscribed users	Log entry created with all correct fields. Push notifications delivered to 2 subscribers	PASS	
6	Notifications saved and admin UI updates	Records inserted in `user_checkpoint_notifications` table. Success snackbar appears: "تم تحديث الحالة ✅", card status updates to purple "	Notification records saved. Admin UI updated instantly showing purple "حاجز" without refresh	PASS	
7	Regular users receive real-time updates	The user viewing checkpoint list sees status change to "حاجز" instantly. Push notification appears in device notification tray	User app updated in real-time showing "حاجز". Push notification received with sound	PASS	
8	User taps notification and admin checks history	User's app opens to قلنديا page showing "حاجز" status. Admin views "سجل التحديثات" page showing log entry	App opened to correct checkpoint with updated status. History log shows: "أحمد غير حالة قلنديا من مفتوح (داخل) إلى حاجز"	PASS	

Table 11 : Test Case 3 For Reports & Update Road Status

Acceptance Testing and User Feedback

Acceptance tests were also conducted to examine how users accept the system by imitating their natural behavior rather than being concerned with inner implementation details of the system. The tests were conducted by nine users of the application in natural Android devices rather than artificial test platforms. During testing, users interacted with some of the important functionalities of the system, such as logging in to the application or viewing the status of the road.

During the acceptance testing phase, feedback from users of the program was obtained and used. Naturally, one of the users was worried about recovering an account in case the password was forgotten. As a result of this feedback, the password was integrated into the program to assist in usability and increased accessibility of the program. After the addition of this improvement, it was realized that the program was still within the acceptance testing criteria.

Chapter 6: Conclusion and Future works

6.1 Conclusion

The "Takkeh" system addresses the problems faced by drivers in Palestine, from sudden road closures to traffic congestion. It gathers real-time traffic reports from crowdsourced sources, such as Telegram groups, and filters them using advanced Natural Language Processing (NLP) techniques to extract accurate and organized information about the status, location, direction, and timing of checkpoints.

The system offers a simple and user-friendly interface, allowing users to view a real-time map of their current location and the locations of reported checkpoints. Users can also update road conditions, and the system verifies these updates through user authentication or administrator approval.

A key component of the system is its routing and notification service, which enables users to subscribe to all checkpoints along their route and receive instant notifications if any checkpoint they pass is closed, reopened, becomes congested, or if an accident occurs. There is also an administrative dashboard that enables supervisors to monitor user activity, review reports, manage system data, and optimize the "NLB" analytics.

Although the team found it challenging to work with unstructured data, maintain accuracy, and obtain reliable information, the project successfully demonstrated the use of artificial intelligence and automation to enhance road safety, raise traffic awareness among Palestinian drivers, and protect them from road hazards through regular system checks.

6.2 Future Works

Enhanced Data Collection and Evaluation

1. Use various data sources that could be obtained through GPS, cameras, as well as road sensors to obtain data in real-time about the checkpoints' statuses and road conditions.

2. Applying advanced Machine Learning to predict levels of traffic congestion, calculate impact of checkpoints, and categorize kinds of roads and obstacles.

Intelligent Transportation System Connectivity

1. Add route optimization tools that provide alternative routes to avoid congested areas and check points.

2. Integrate public transport services such as the integration of the route of a specific bus or taxi service with specific road conditions.

3. Use of long-term data collected from various traffic control points for improving planning in the development of physical infrastructure.

Map and Navigation Enhancements

1. Provide users without a selected route with access to route visualization as well as suggestions.

2. Enhance the current method of sending notifications with the addition of a voice notification method to ensure the student is well guided with minimal distraction.

User Accessibility

It has the capacity for guest browsing in order to further enable the capacity for an individual or person to view the information without necessarily registering.

References

[1]:Wikipedia, "Information Extraction." [Online]. Available: https://en.wikipedia.org/wiki/Information_extraction. [Accessed: Oct. 20, 2025, 14:30].

[2]:A. Author et al., "A Rule-Based Information Extraction System." [Online]. Available: https://www.researchgate.net/publication/XXXX. [Accessed: Oct. 20, 2025, 16:21].

[3]: FADA – Birzeit University Institutional Repository, "The Wall, Bypass Roads and the Dual Transportation System in Palestine." [Online]. Available: https://fada.birzeit.edu. [Accessed: May 18, 2025, 20:35].

[4]: M. Calì and S. Miaari, "The Labor Market Impact of Mobility Restrictions: Evidence from the West Bank," *SSRN*, 2018.Available:[Google Scholar]

[5]: Y. Liu et al., "USTC-NELSLIP at SemEval-2022 Task 11: Gazetteer-Adapted Integration Network for Multilingual Complex Named Entity Recognition," arXiv:2203.03216, 2022. [Online]. Available: https://arxiv.org/abs/2203.03216. [Accessed: Oct. 20, 2025, 18:40].

[6]Supabase, "Supabase Documentation." [Online]. Available: https://supabase.com. [Accessed: Oct. 20, 2025].

[7]Google, "Firebase Console." [Online]. Available: https://console.firebase.google.com. [Accessed: Oct. 20, 2025].

[8]"Driving directions, live traffic & road conditions updates," Waze. [Online]. Available: https://www.waze.com

[9]S. Vajjala, B. Majumder, A. Gupta, and H. Surana, *Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems*. O'Reilly Media, 2020. Accessed: Oct. 21, 2025. [Online]. Available: https://books.google.ps

[10]"Waze navigation & live traffic," *App Store*. [Online]. Available: https://apps.apple.com

[11] "أحوال طرق فلسطين," *App Store*. [Online]. Available:https://apps.apple.com

[12] "ع وين رايح؟," *App Store*. [Online]. Available:https://apps.apple.com

[13] "Azmeh أزمة," *App Store*. [Online]. Available:https://apps.apple.com

[14]"Telegram messenger," *App Store*. [Online]. Available:https://apps.apple.com

[15]H. Aburas, I. Shahrour, and M. Sadek, "Leveraging crowdsourcing for mapping mobility restrictions in data-limited regions," *MDPI*, 2024. [Online]. Available:[Google Scholar]

[16]T. El Moussaoui and C. Loqman, "Advancements in Arabic named entity recognition: A comprehensive review," *IEEE Access*, vol. 12, pp. 172156-172178, 2024, doi: 10.1109/ACCESS.2024.3491897.Available:[Google Scholar]

[17] M. Khalilia, S. Malaysha, R. Suwaileh, M. Jarrar, A. Aljabari, T. Elsayed, and I. Zitouni, "ArabicNLU 2024: The first Arabic natural language understanding shared task," *arXiv preprint arXiv:2407.20663*, 2024. [Online]. Available: https://arxiv.org/abs/2407.20663.Available:[Google Scholar]

[18]F. Sufi and I. Khalil, "Automated disaster monitoring from social media posts using AI based location intelligence and sentiment analysis," Mar. 2022. [Online].Available:[Google Scholar]

Appendix

Use Cases (Full Use Cases)

Use-Case 1: User Registration, Login and Authentication

1. Brief Description

This use case describes how users register and log in to the Takkeh application using email-based verification. New users must verify their email address through a verification link before completing their registration by creating a username and password. All users are initially registered as regular users. Administrative users are identified based on their presence in the admin table after being promoted from a regular user account.

2. Actor

2.1 Actors

2.1.1 New User: A person registering in the app for the first time.

2.1.2 Registered User :A person who already has an account using a username and password.

2.1.3 Admin User: A user who exists in the admin table and has elevated privileges.

3.Preconditions (Entry Condition)

3.1 The Takkeh application is installed on the user's device.

3.2 The device has an active internet connection.

3.3 The user has a valid email address.

4. Flow of Events

4.1 Basic Flow – New User Registration

4.1.1 The user opens the Takkeh application.

4.1.2 The user selects the "Register" option.

4.1.3 The system asks the user to enter their email address (must be unique).

4.1.4 The system checks if the email is already registered:

a. If yes → show error: "This email is already registered."
b. If not → the system sends a verification link to the provided email.

4.1.5 The user opens the email and clicks the verification link.

4.1.6 The system verifies the link:

a. If invalid or expired → show error: "Invalid verification link."
b. If valid → proceed to the next step.

4.1.7 The system displays a registration form where the user must enter: A unique username,a password (must meet security requirements: minimum 8 characters, include letters and numbers) and confirm password (must match the password).

4.1.8 The system validates: Username uniqueness, password strength, password and confirm password match.

a. If not valid → show appropriate error message.
b. If all data is valid → store the new user in the users table with a securely hashed password.

4.1.9 The system redirects the user to the Login page.

4.2 Alternative Flow – Registered User and Admin Login

4.2.1 The user selects "Log In".

4.2.2 The user enters username/email and password.

4.2.3 The system verifies the provided credentials:

If username is not found → show error.

If the password is incorrect → show error.

4.2.4 If the credentials are valid:

If the user exists in the users table only → redirect to the User dashboard.

If the user exists in both users and admin tables → the system prompts:

Do you want to enter as Admin or User?"

- If Admin → redirect to Admin dashboard
- If User → redirect to User dashboard

4.2.5 The user is redirected to the selected dashboard.

5. Post-conditions

5.1 A user is stored in the database with a verified email address and credentials.

5.2 user_id is the primary key, while the email address and username are unique.

5.3 Admin users are recognized through the admin table.

5.4 Passwords are hashed securely.

6. Special Requirements

6.1 Verification link expires within a limited time window.

6.2 Username and email must be unique.

6.3 The password must meet security policies (minimum 8 characters, letters and numbers).

6.4 All actions must complete within 5 seconds under normal network conditions.

6.5 Admin requests must be secure and logged.

7. Extension Points

7.1 Email Verification**:** Invalid/expired verification links show an error message, and the user can request a new link.

7.2 System Error: Show message: "Something went wrong. Please try again later."

Use Case 2: Submit and Update Road Status

1. Brief Description

This use case describes how a registered user can update the road's status such as open, closed, accident,checkpoint and traffic. Once five unique users submit the same updated status for the same road within a time period of 30 minutes or less, the system auto-updates the status. Alternatively, an administrator can directly approve user updates or manually change road statuses.

2. Actor

2.1 Actors

2.1.1 **Registered User:** A user with an active account who can submit road updates.

2.1.2 **Admin User:** A privileged user who can approve or reject user updates or manually change road statuses.

3. Preconditions (Entry Conditions)

3.1 The registered user is logged into the Takkeh application with a valid account.

3.2 The system has access to the internet and the database is operational.

3.3 For admin actions, the logged-in user must have administrator privileges.

4. Flow of Event

4.1 Basic Flow – User Updates Road Status

4.1.1 The registered user selects the "Update Road Status" option for a specific road.

4.1.2 The user selects the new status like Open , Closed, traffic, accident and checkpoint.

4.1.3 The system logs the user's update along with their user ID.

4.1.4 The user receives a message : "Your update has been submitted ."

4.1.5 The system checks the number of updates for the same status on the same road.

4.1.6 The system checks if five different users have reported the same road condition within 30 minutes or less. If the condition is met, the system automatically updates the road condition and records the change as an "auto-update" with the location ID.

4.2 Alternative Flow – Admin Approves User Update

4.2.1 If a user submitted an update then a notification will be sent to all admins.

4.2.2 The admin user accesses the "Notifications" section.

4.2.3 The admin reviews user-submitted updates for a road or checkpoint.

4.2.4 The admin clicks "Approve" for a user update then the system immediately updates the road status to the approved status.

4.2.5 The admin may also choose to "Reject" a user-submitted update. Rejected updates are logged and removed from the pending queue.

4.2.6 The manual approval is logged with admin ID, user ID, checkpoint ID, timestamp, direction and old vs new status values.

4.3 Alternative Flow – Admin Manually Changes Road Status

4.3.1 The admin user selects a road or checkpoint.

4.3.2 The admin manually changes the status like from Open to Closed.

4.3.3 The system immediately updates the road status in the database.

4.3.4 The update is logged as an "Admin Change" for record-keeping.

4.3.5 The manual change is logged with admin ID, checkpoint ID, timestamp, direction and old vs new status values.

5. Post-conditions

5.1 If the user update contributed to the threshold, it is logged and counted.

5.2 If five valid unique user reports within 30 minutes or less are collected, the

road status is updated.

5.3 Admin-approved updates are applied immediately.

5.4 Manual admin changes override any pending user reports.

6. Special Requirements

6.1 The system prevents users from submitting multiple updates for the same checkpoint within a 30-minute cooldown period.

6.2 Each update (user or admin) is stored with a user/admin ID, checkpoint ID, timestamp, direction and old vs new status values.

6.3 The interface must clearly distinguish statuses changed by automatic update (user reports), manual admin change or admin-approved user report.

7. Extension Points

7.1 Multiple Reports Extension

If the user attempts to submit another update for the same checkpoint within 30 minutes, the system prevents the submission.

7.2 Admin Notification Extension

The system periodically checks for pending user updates and sends a notification to the admin dashboard every 5 minutes if new pending updates exist.

Use-Case 3: Searching for Specific Roads Status

1. Brief Description

This use case describes a registered user interacting with the system to search for information about specific roads or checkpoints. The system processes the input, fetches data from the database, and returns relevant results. It also handles cases when no results are found.

2. Actor

2.1 **Actor** :Registered User: A user with an active account who can search for road information.

3. Preconditions (Entry Conditions)

3.1 The user is logged in to the Takkeh application using a valid account.

3.2 The system's database is online and contains searchable road/checkpoint data.

4. Flow of Events

4.1 Basic Flow – Search for a Road or Checkpoint

4.1.1 The user selects the "Search" option from the home interface.

4.1.2 The user enters the name or location of the checkpoint.

4.1.3 The system searches the database for matching results.

4.1.4 The system displays a list of results showing road names, current statuses such as open, closed, traffic , accident, checkpoint, and last updated times for each direction “Inbound & Outbound”.

4.2 Alternative Flow – No Search Results Found

4.2.1 The user enters a keyword or location that doesn’t match any entry.

4.2.2 The system shows a message: “No results found. Please try another search.”

4.3 Alternative Flow – Database Offline or No Internet

4.3.1 The user attempts a search while the system is offline.

4.3.2 The system displays an error message: "Unable to connect. Please check your internet connection or try again later."

5. Post-conditions

5.1 The user successfully views the details of the searched city or checkpoint.

6. Special Requirements

6.1 The search should return results within 3 seconds.

6.2 Partial or fuzzy search must be supported.

7. Extension Points

7.1 Auto-Suggestions List

While the user types in the search field, the system displays a dynamic list of matching provinces and checkpoints. The user can select an item from the list to open its details directly.

Use-Case 4: Telegram Road Status Processing with NLP

1. Brief Description

In this use case, the Takkeh system automatically collects real-time road status updates from designated Telegram groups/supergroups. These messages are initially saved as raw, unprocessed data and then analyzed using a local rule-based Natural Language Processing (NLP) pipeline.

The NLP module applies a combination of text normalization, gazetteer-based location extraction, rule-based status classification, direction detection, and historical pattern enhancement to convert unstructured Telegram messages into structured road status reports.

The extracted information—such as checkpoint location, road status, direction (inbound/outbound/both), confidence score, and timestamp—is stored in the database and reflected in the Takkeh mobile application as real-time road update.

2. Actor

2.1 Actor : Telegram Traffic Groups/Supergroups.

3. Preconditions (Precondition)

3.1 The Takkeh system connects to specified Telegram groups with a valid BOT_TOKEN.

3.2 The Telegram bot is allowed to read messages in group or supergroup chats.

3.3 The database (Supabase) is accessible and editable.

3.4 The cities table is completed and serves as the location gazetteer.

3.5 Location synonyms are configured (manual + auto-generated with prefixes like "حاجز","مفرق").

3.6 Historical checkpoint patterns data exists for pattern-based confidence enhancement.

3.7 The NLP processor can retrieve newly ingested raw Telegram messages.

4. Flow of Events

4.1 Basic Flow - Automated Message Processing

4.1.1 A new road-status message is posted in a monitored Telegram group/supergroup.

4.1.2 The NLP processor retrieves unprocessed messages from storage (processed=false).

4.1.3 Messages are ordered temporally to support time-window-based grouping..

4.1.4 Messages are grouped by time proximity:

a. Messages within 20 seconds are combined
b. Maximum 4 fragments per group
c. Fragmented messages are merged into coherent updates

4.1.5 Text normalization is applied to each message group:

a. Arabic diacritics removal (تشكيل)
b. Letter variants standardization (إ أ آ → ا, ى → ي, ة → ه)
c. Hamza normalization (ؤ → و, ئ → ي)
d. Whitespace cleanup

4.1.6 Intent detection filters non-informative messages:

a. Question-like messages are detected (e.g., “?”, “شو“ ,”كيف“ ,”وين الوضع”) and skipped to avoid corrupting road status.
b. Messages with no valid detected location are classified as low-value

and discarded.

c. Messages with a valid location but unclear status are still processed and stored as **unclear**.

4.1.7 Location extraction is performed using a gazetteer-based approach:

a. Exact matching (highest priority, supports multi-word locations).
b. Automatic synonym resolution ("قلنديا" → "حاجز قلنديا")
c. Fuzzy matching using character 2-gram Jaccard similarity (threshold ≥ 0.72).
d. Multiple locations per message are supported
e. Standalone word validation (ensures location is not part of another word)
f. Minimum location length enforcement (4+ characters for exact match)

4.1.8 For messages with multiple locations:

a. Text is segmented using Arabic language separators (.، :، !، و،؟، ،)
b. Each location receives its own contextual segment (±40 chars)
c. Each location is analyzed independently.

4.1.9 Direction scope is determined for each location:

a. Inbound hints: داخل، جنوب، طالع، رايح، دخول
b. Outbound hints: خارج، شمال، نازل، راجع، خروج
c. Both directions: بالاتجاهين، داخل وخارج، رايح وراجع
d. Default scope: both (if unclear)
e. Context slicing around direction keywords (±30 chars)

4.1.10 Road status is classified using weighted rule-based patterns:

a. **Contextual patterns** (priority, weight 0.88):
    - checkpoint: سيارة سيارة، وحدة وحدة، على مهلة، تفتيش بطيء
    - traffic: شوي شوي، ببطء، طابور طويل، نص ساعة انتظار
    - open: بتمشي منيح، تمام التمام، الحمد لله، بدون مشاكل

b. **Standard patterns**:
   - closed (1.0): مغلق، مسكر، ❌، مقفول، ممنوع
   - accident (0.95): حادث، تصادم، انقلاب، اصابة
   - traffic (0.9): ازمة، زحمة، ازدحام، طابور، واقف
   - checkpoint (0.6): حاجز، ⚠️، كمين، بوابة، تفتيش
   - open (0.85): سالك، ✅، مفتوح، ماشي، فاضي
c. **Negation handling**:
   - Negation detection (مش، ما، غير، لا، بدون)
   - Status inversion if negation found ("مش مغلق" → open)
d. **Emoji-based hints**: ✅ = open, ❌ = closed , ⚠️ = checkpoint

4.1.11 Confidence score is calculated for each location:

a. Base confidence for valid location presence.
b. Weighted contribution from matched status patterns.
c. Contribution from gazetteer match quality.
d. a historical pattern-based adjustment in the range ±0.15 is applied using checkpoint-specific statistics from the last 30 days.
e. The final confidence score is normalized to the range [0,1].

4.1.12 Historical pattern enhancement is applied when historical data exists (typical status trends, common keyword alignment, and distribution matching from recent history).

a. Typical status patterns (usually_open/usually_closed): ±0.08
b. Common keyword matching (top 10 keywords): +0.04
c. Status distribution alignment: +0.03
d. Pattern data from last 30 days
e. Total boost range: ±0.15

4.1.13 The final structured output is stored and published as a live road-status update visible to users in the app, and also accessible in the admin dashboard for monitoring.

4.1.14 The raw message is marked as processed (processed=true) to prevent reprocessing

4.2 Alternative Flow - Unprocessable Message

4.2.1 If the message contains no valid detected locations (after exact and fuzzy matching), it is classified as low-value.

4.2.2 If the message is a question, it is skipped to avoid generating incorrect updates.

4.2.3 The system marks the message as processed and records the outcome for monitoring, without generating a road update.

5. Post-conditions

5.1 The database contains relevant traffic information.

5.2 Upon processing, user-visible relevant traffic is presented as a part of normal app use.

5.3 The irrelevant or unprocessable messages have been discarded (or saved) for future enhancements.

5.4 The app refreshes the update section automatically to include the newly processed information.

5.5 System performance and accuracy metrics are recorded for monitoring and optimization.

5.6 System performance metrics are saved to nlp_performance_metrics for monitoring NLP accuracy

5.7 All processed messages are marked with processed=true to avoid reprocessing

6. Special Requirements

6.1 The NLP module must support:

a. Arabic language (including Palestinian dialect variations)
b. Contextual pattern recognition with weighted scoring
c. Multi-location extraction per message
d. Direction-aware status classification (inbound/outbound/both)
e. Negation handling with status inversion
f. Historical pattern learning and confidence adjustment

6.2 Batch processing time is typically under 8 seconds under normal conditions..

6.3 All messages must be retrieved securely via available API keys.

6.4 The system will record all actions attempted related to the processing of the attempts for performance monitoring.

6.5 Message grouping window: 20 seconds, maximum 4 fragments per group

6.6 Fuzzy matching similarity threshold: 72%

6.7 Historical pattern data window: 30 days

7. Extension Points

7.1 Error Handling - API or NLP Errors

If a message cannot be processed due to NLP ambiguity, system errors, or external API failures, the system records the failure details for monitoring and analysis. Failed attempts are counted as part of system performance evaluation, and the message is safely excluded from repeated processing to avoid infinite retry loops. Such cases are preserved for offline inspection and future improvement of the NLP pipeline.

7.2 Performance Monitoring

The system records detailed metrics for each processing batch:

a. Messages processed, saved, failed
b. Questions skipped, low-value skipped
c. Average/min/max confidence scores
d. Confidence distribution (very_low/low/medium/high/very_high)
e. Locations detected, multi-location messages
f. Status breakdown by type (open/closed/traffic/checkpoint/accident)
g. Pattern boosts applied count and average boost value
h. Processing time in milliseconds